A physics-informed neural network and a Fourier neural operator, head to head:

steady inviscid flow over a family of NACA four-digit airfoils, scored against SU2

Eder Sousa

October 2026

Abstract

Three learned models are compared with SU2, on steady two-dimensional inviscid compressible flow around NACA four-digit sections. The models are a Fourier neural operator (FNO), a proper orthogonal decomposition (POD) baseline and a physics-informed neural network (PINN). The input space is Mach number 0.30–0.80, angle of attack $-2°$ to $6°$, thickness 0.08–0.16 and camber 0–0.04, with an extrapolation set at Mach 0.80–0.85. SU2 ran 1,184 Euler cases on a structured O-grid. The FNO and POD are trained on up to 896 of them; the PINN is trained on each test case separately, from the equations alone. Every contestant is scored on the same nodes against SU2 on a 512×193 grid, with one force integrator. Each error is shown beside *the floor*: SU2's own 256×97 solution scored the same way. In range, the FNO's near-field error is 1.3 times the floor at its 128×49 training resolution and 5.3 times at 256×97 on subcritical cases, and 2.4 times on supercritical cases. In extrapolation every surrogate fails: the FNO is 3.6 times the floor and POD 7.9 times. The PINN was first built on the Euler equations, and that version converged to fluid at rest. It was replaced by a full-potential PINN, which is 26–65 times the floor and under-predicts lift on five of six cases. On cost, the PINN is 88–220 times SU2 per case, depending on how GPU and CPU time are counted. The FNO repays its training only after 1,126 new cases. Three predictions were recorded before any model ran. The first held on wall time only; the second held; the third held in part. All of this is relative to SU2 at 512×193. On the NACA 0012 anchor, that truth is itself 6–8 % below the grid-converged lift of Vassberg and Jameson.

Contents

1 Introduction **3**

2 Problem formulation **4**
2.1 Governing equations 4
2.2 Geometry 5
2.3 The O-grid 5

3 Reference data: SU2 **6**
3.1 Solver settings and the convergence rule 6
3.2 Stage S0: accuracy and cost 7
3.3 Stage S1: the data campaign 8

4 The learned models **9**
4.1 POD baseline . . . 9
4.2 Fourier neural operator . . . 10
4.3 PINN . . . 11
4.3.1 The Euler PINN, and why it converges to fluid at rest . . . 12
4.3.2 The full-potential PINN . . . 12

5 Scoring **14**

6 Results **16**
6.1 The three predictions . . . 16
6.2 Accuracy by regime and set . . . 16
6.3 Fields and wall pressure . . . 17
6.4 Lift . . . 18
6.5 PINN training . . . 18
6.6 FNO: seeds, loss B and shocks . . . 19
6.7 Data efficiency and break-even . . . 20
6.8 Physics checks and failures . . . 21

7 Discussion **22**
7.1 What the comparison does and does not show . . . 24
7.2 Limitations . . . 25

8 Open items and reproducibility **25**

A Neural networks from the ground up **26**
A.1 A network is a parameterised function . . . 26
A.2 Training is optimisation . . . 27
A.3 Automatic differentiation . . . 28
A.4 Optimisers . . . 28
A.5 Generalisation, and what it means for each model . . . 29

B The Fourier neural operator in detail **30**
B.1 Learning an operator . . . 30
B.2 From kernel integrals to Fourier multipliers . . . 30
B.3 The Fourier layer as implemented . . . 30
B.4 Working in the O-grid's index space . . . 31
B.5 Inputs and outputs . . . 32
B.6 Training . . . 32
B.7 What truncation does to a shock . . . 33
B.8 Evaluation, and keeping the test set out . . . 33

C The physics-informed neural network in detail **35**
C.1 The idea . . . 35
C.2 The network and its inputs . . . 35
C.3 Hard and soft constraints . . . 35
C.4 Derivatives through the grid map . . . 35
C.5 Collocation points . . . 36
C.6 The loss and its weights . . . 38

C.7 Optimisation 38
C.8 What this study teaches about PINNs 38

D The three models side by side **39**

1 Introduction

The two families of learned flow solvers make opposite bets.

- A *neural operator* learns a map from problem parameters to solution fields across a whole family of cases [5]. It pays a large data and training cost once and then answers new cases almost for free.
- A *physics-informed neural network* [8] needs no data. It is trained on one case by minimising the residual of the governing equations at collocation points, and it pays the full training cost again for every case.

This study measures both bets on one aerodynamic problem, against one truth, with one scorer and a cost record for every method.

The problem. The problem is steady inviscid compressible flow (the Euler equations) around NACA four-digit sections [1], over a box of four inputs: free-stream Mach number M_∞, angle of attack α, thickness t and maximum camber m. The range runs from fully subcritical flow to transonic flow with strong shocks. It is cheap enough that a laptop can generate a thousand reference solutions with SU2 [2], and hard enough to exercise everything that makes compressible aerodynamics difficult for learned models:

- shocks that move with the inputs;
- lift fixed by a condition at a sharp trailing edge;
- forces that depend only on the pressure along the wall.

The contestants.

1. **SU2** on the 256×97 training grid. This is the reference method, and scored against the fine grid it defines *the floor*: the error any 256×97 answer carries by being on that grid.
2. **POD**: a linear basis of training fields, with mode weights interpolated over the inputs.
3. **FNO**: a Fourier neural operator in the O-grid's index space.
4. **PINN**: trained on each test case from the equations, boundary conditions and that case's inputs only.

Both networks are written from scratch in PyTorch.

Predictions recorded before any model ran.

P1. For a single new case, SU2 beats the PINN on cost by one to two orders of magnitude.

P2. The FNO beats "just run SU2" only after roughly as many new cases as it took to train it, about a thousand, unless the data-efficiency curve shows far fewer training cases suffice.

P3. Without the Kutta condition, the PINN's residuals will look acceptable while its lift is wrong.

These are checked in Section 6.1.

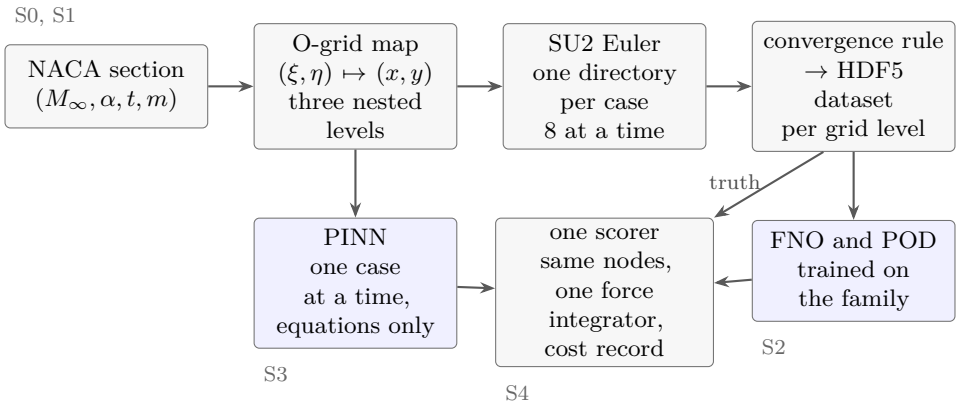


Figure 1: The pipeline. One function defines the geometry: the mesh generator samples the O-grid map and the PINN uses it continuously. The models never see the truth, which only the scorer reads.

For readers new to the methods. Appendices A to D explain the learned models in detail, for teaching. They cover the network building blocks and their training, the FNO as implemented, the PINN as implemented, and the three models side by side.

How the work ran. The work ran in five stages (Figure 1):

- S0 checked SU2's accuracy and cost, and froze its settings;
- S1 ran the data campaign;
- S2 trained the operator models;
- S3 ran the PINN;
- S4 scored everything.

Every figure quoted here comes from a committed record in `results/`. The dated log `docs/findings.md` gives the command behind each figure.

2 Problem formulation

2.1 Governing equations

The flow obeys the steady two-dimensional Euler equations in conservation form,

$$\frac{\partial \boldsymbol{F}}{\partial x} + \frac{\partial \boldsymbol{G}}{\partial y} = 0, \qquad \boldsymbol{F} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ \rho uH \end{pmatrix}, \quad \boldsymbol{G} = \begin{pmatrix} \rho v \\ \rho uv \\ \rho v^2 + p \\ \rho vH \end{pmatrix}, \tag{1}$$

for a perfect gas with $\gamma = 1.4$, where

$$p = (\gamma - 1)(\rho E - \tfrac{1}{2}\rho |\boldsymbol{u}|^2), \qquad H = E + p/\rho. \tag{2}$$

Boundary conditions. On the wall the flow is tangent to the surface, $\boldsymbol{u} \cdot \hat{\boldsymbol{n}} = 0$. On a circle of radius 100 chords it meets a uniform free stream at Mach M_∞ and angle α. SU2 runs dimensionally, with $p_\infty = 101{,}325\,\mathrm{Pa}$, $T_\infty = 288.15\,\mathrm{K}$ and $R = 287.058\,\mathrm{J/(kg\,K)}$.

Non-dimensional state. Every array a model sees is non-dimensional,

$$\boldsymbol{q} = (\hat{\rho},\ \hat{u},\ \hat{v},\ \hat{p}) = (\rho/\rho_\infty,\ u/U_\infty,\ v/U_\infty,\ p/p_\infty). \tag{3}$$

The derived quantities used throughout come from one tested module:

$$C_p = \frac{\hat{p}-1}{\frac{1}{2}\gamma M_\infty^2}, \qquad M = M_\infty \frac{|\hat{\boldsymbol{u}}|}{\sqrt{\hat{p}/\hat{\rho}}}, \tag{4}$$

$$\frac{s-s_\infty}{c_v} = \ln\frac{\hat{p}}{\hat{\rho}^{\,\gamma}}, \qquad \frac{H}{H_\infty} = \frac{\hat{p}/\big(\hat{\rho}(\gamma-1)M_\infty^2\big) + \frac{1}{2}|\hat{\boldsymbol{u}}|^2}{1/\big((\gamma-1)M_\infty^2\big) + \frac{1}{2}}. \tag{5}$$

Two identities make these useful checks:

- In shock-free inviscid flow the entropy must stay at its free-stream value, so any rise is numerical error.
- In steady inviscid flow from a uniform stream the total enthalpy H is constant everywhere, through shocks included.

2.2 Geometry

A NACA four-digit section of unit chord is parameterised as one closed curve of $\xi \in [0,1)$: $\xi = 0$ is the trailing edge, $\xi \in [0, \frac{1}{2}]$ runs along the lower surface to the leading edge, and $\xi \in [\frac{1}{2}, 1)$ runs back along the upper surface.

Thickness. With $s = \cos\pi\xi$, the chord station is $x = s^2$, so $\sqrt{x} = |s|$ and the curve has no square-root singularity at the leading edge. The signed half-thickness is

$$y_s(\xi) = -\operatorname{sign}(s)\, 5t \left(0.2969\sqrt{x} - 0.1260x - 0.3516x^2 + 0.2843x^3 - 0.1036x^4\right), \tag{6}$$

where the x^4 coefficient closes the trailing edge.

Camber. The camber line, with $p = 0.4$ fixed, is

$$y_c = \begin{cases} \dfrac{m}{p^2}\left(2px - x^2\right), & x < p, \\ \dfrac{m}{(1-p)^2}\left(1 - 2p + 2px - x^2\right), & x \geq p. \end{cases} \tag{7}$$

The thickness is applied normal to it: the wall point is $(x - y_s\sin\theta_c,\ y_c + y_s\cos\theta_c)$, with $\tan\theta_c = \mathrm{d}y_c/\mathrm{d}x$.

The same function runs on NumPy arrays and on PyTorch tensors, so the mesh and the PINN share one geometry.

2.3 The O-grid

All data live on one structured O-grid, defined as a continuous map $(\xi, \eta) \in [0,1) \times [0,1] \mapsto (x, y)$:

$$\boldsymbol{x}(\xi,\eta) = \boldsymbol{x}_w(\xi) + \sigma(\eta)\left(\boldsymbol{x}_f(\xi) - \boldsymbol{x}_w(\xi)\right), \qquad \boldsymbol{x}_f(\xi) = \left(\tfrac{1}{2} + R\cos(-2\pi\xi),\ R\sin(-2\pi\xi)\right), \tag{8}$$

with $R = 100$ chords. The radial distribution is geometric,

$$\sigma(\eta) = \frac{r^{N\eta} - 1}{r^N - 1}, \qquad N = 192, \tag{9}$$

where the ratio r is chosen so that the first wall spacing of the 256×97 grid is 2×10^{-3} chords.

Table 1: The input box, the sets and the grids, all fixed before any model was trained.

Inputs		
Mach number M_∞	$[0.30,\ 0.80]$	extrapolation set: $[0.80,\ 0.85]$
Angle of attack α	$[-2°,\ 6°]$	
Thickness t	$[0.08,\ 0.16]$	
Maximum camber m	$[0,\ 0.04]$	camber position $p = 0.4$ fixed
Sets (scrambled Sobol designs, seeds fixed)		
Training	Sobol points 1–896	256×97
Validation	Sobol points 897–1,024	256×97; tuning and early stopping only
Test, in range	64 points, another seed	512×193 (truth) and 256×97 (floor)
Test, extrapolation	16 points, $M_\infty \in [0.80, 0.85]$	512×193 and 256×97
Grids (nested exactly: each level is every second node of the next)		
Fine	512×193	truth
Training	256×97	data, evaluation, the floor
Coarse	128×49	FNO training

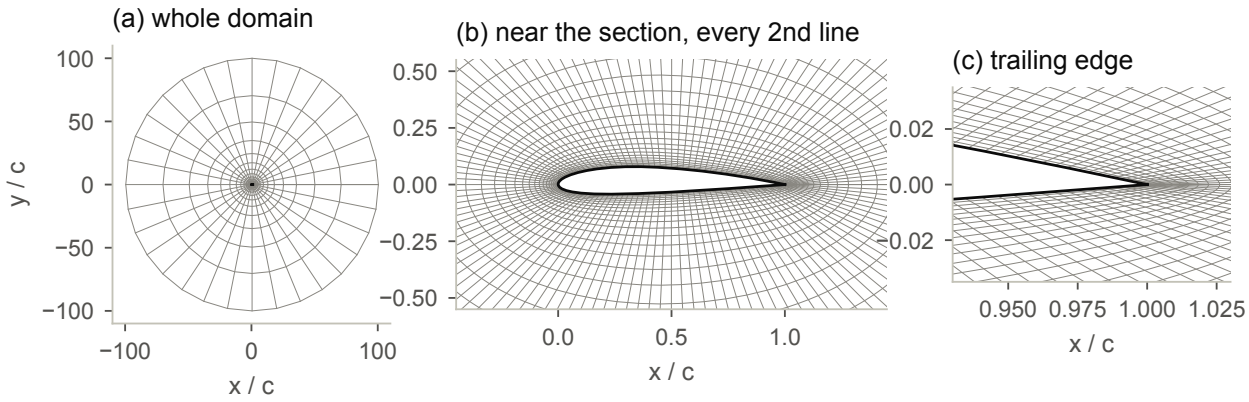


Figure 2: The 256×97 O-grid around a $t = 0.12$, $m = 0.02$ section. (a) The whole domain, every eighth line around and every fourth outward. (b) Near the section. (c) The trailing edge, where the section's cosine clustering crowds the lines and makes $\partial \boldsymbol{x}/\partial \xi$ vanish at the edge itself: $x_\xi = y_\xi = 0$, so the map is degenerate there ($J = 0$). That point matters for the PINN (Section 4.3.2).

Nested levels. The grids sample the map at $\xi_i = i/N_I$ and $\eta_j = j/(N_J - 1)$. Each coarser level is therefore exactly every second node of the next, which lets the truth be read at the training grid's nodes without interpolation. Node (i, j) has SU2 id $jN_I + i$, so every output maps straight back onto an (i, j) array.

The outer point. The outer point sits at the uniform polar angle $-2\pi\xi$. The first construction instead placed it at the same angle about mid-chord as the wall point. That version folded the mesh: 9,792 inverted cells at $t = 0.08$, $m = 0.04$. The uniform angle has no inverted cell at any corner of the box (Figure 2).

3 Reference data: SU2

3.1 Solver settings and the convergence rule

Settings. SU2 8.5.0 solves Equation (1) on each case's own mesh, one serial process per case:

- a JST scheme [3] with sensor coefficients $(0.5, 0.02)$;
- implicit Euler pseudo-time stepping with an adaptive CFL;
- an FGMRES linear solver with ILU preconditioning;
- weighted-least-squares gradients.

The convergence rule. Convergence is judged by a rule of our own, read from the history file, not by SU2's flag. A case is *converged* if and only if both hold:

$$\log_{10} \frac{\mathrm{rms}(R_\rho)_1}{\mathrm{rms}(R_\rho)_N} \geq 5 \quad \text{and} \quad \max_{k \in \mathcal{W}} C_\bullet^{(k)} - \min_{k \in \mathcal{W}} C_\bullet^{(k)} \leq 10^{-6} \ \text{ for } C_\bullet \in \{C_L, C_D\}. \tag{10}$$

Here $\mathcal{W}$ is the last 500 iterations. The caps are 10,000 iterations on the training grid and 20,000 on the fine grid.

Consistency with SU2's own stop. SU2 stops when the mean relative change of C_L and C_D over n_c iterations falls below ε_c. At that stop C_L can still vary by up to $\varepsilon_c n_c |C_L|$ over the window. The frozen setting therefore has to satisfy

$$\varepsilon_c \, n_c \, |C_L|_{\max} = 10^{-9} \times 500 \times 1.86 = 9.3 \times 10^{-7} < 10^{-6}, \tag{11}$$

where $|C_L|_{\max} = 1.86$ is a thin-airfoil bound at the box's highest-lift corner. Stage S0 found this the hard way:

- with $\varepsilon_c = 10^{-7}$, none of 24 runs met Equation (10);
- with 10^{-8}, 9 of 23 did;
- with 10^{-9}, all 24 did.

3.2 Stage S0: accuracy and cost

Five cases were each solved on all three grids:

- a symmetric section at $\alpha = 0$;
- two subcritical lifting cases;
- a supercritical case at $M_\infty = 0.75$;
- the NACA 0012 anchor at $M_\infty = 0.8$, $\alpha = 1.25°$.

Multigrid. Three multigrid levels converged every case and agreed on C_L to 3×10^{-9}. They cost more wall time on all five cases (median 239.8 s against 132.6 s), so the campaign runs without multigrid.

Mesh error. Writing f_c, f_t and f_f for the coarse, training-grid and fine values, the observed order is

$$\hat{p} = \log_2 \frac{|f_c - f_t|}{|f_t - f_f|}. \tag{12}$$

It was negative or non-monotone for C_L on every lifting case. The three grids are therefore not in the asymptotic range, and the error is reported as the plain difference from the fine grid (Table 3), not as a Richardson extrapolation.

Table 2: The NACA 0012 anchor ($M_\infty = 0.8$, $\alpha = 1.25°$): this study's SU2 grids against the four continuum estimates of Vassberg and Jameson [10] (their Table 11). C_M is compared in magnitude; both are nose-down.

	This study, SU2			Vassberg & Jameson continuum	
	128×49	256×97	512×193	min	max
C_L	0.3118	0.3174	0.3260	0.3482	0.3562
C_D (counts)	215.2	208.8	212.3	224.5	226.8
$\lvert C_M \rvert$	0.0278	0.0288	0.0311	0.0374	0.0388

Outer boundary. Moving the outer boundary from 100 to 200 chords changed C_L by 1.3×10^{-3} and 2.1×10^{-3}. The radius was kept at 100 chords, so all methods share that far-field truncation.

Known answers.

- The symmetric case gives $|C_L| = 2 \times 10^{-15}$.
- Subcritical drag, ideally zero, is -0.74 to $+1.23$ counts.
- The lift slope is 1.129 times the Prandtl–Glauert thin-airfoil value, above it as a 12 % thick section should be.
- The NACA 0012 anchor gives $C_L = 0.3260$ at 512×193, which is compared with the grid-converged reference in Table 2.

The anchor against a grid-converged reference. Vassberg and Jameson [10] solved this case on aspect-ratio-one O-meshes up to 4096×4096 cells, about 150 chords deep, with three solvers. Their Richardson continuum estimates are $C_L = 0.3482$–0.3562 across the solvers (Table 2). Their section is the NACA 0012 equation with its standard x^4 coefficient, extended to a sharp trailing edge at $x = 1.0089$; ours closes the edge at $x = 1$. Our grids give coefficients below that range on every quantity:

- at 512×193, C_L is 6–8 % low, C_D 12–15 counts low, and $|C_M|$ 17–20 % low;
- the lift is still rising with refinement, from below, while their FLO82 sequence descends to the continuum from above.

So the truth used for scoring carries its own discretisation error on transonic lifting flow. On this case it is about 2.6 times the floor's grid difference in C_L. The comparison among contestants is unaffected, because all of them meet the same truth. Statements such as "near the floor" measure agreement with SU2 at 512×193, not with the exact Euler solution. The differences in section, far field (100 against about 150 chords, and no point-vortex correction here), grid and scheme are not separated.

The scorer check. The scorer's own force integrator (Section 5) reproduced SU2's reported C_L, C_D and C_M on all 24 S0 solutions to 4.9×10^{-11}, against the required 10^{-6}.

3.3 Stage S1: the data campaign

The runs. The campaign ran from 1 to 2 October 2026 with eight concurrent SU2 processes. It ran 1,104 cases at 256×97 (69.0 CPU-hours) and the 80 test cases again at 512×193 (42.3 CPU-hours). The median case took 198.6 s of CPU time at 256×97 and 1,501.7 s at 512×193. A

Table 3: S0: the training grid minus the fine grid (the floor, on the S0 cases), and the cost per case. C_D is in counts (10^{-4}); C_p is compared at the fine grid's every second wall point. All five cases have $t = 0.12$. Cost per case, median over the five cases with six concurrent workers, in seconds of wall / CPU time: 23.7 / 18.0 at 128×49, 132.6 / 127.0 at 256×97, and 1,248.6 / 1,124.2 at 512×193.

Case	M_∞	α	m	ΔC_L	ΔC_D (counts)	ΔC_M	C_p L2	C_p max
sym	0.50	0°	0	1.2×10^{-14}	−0.13	1.5×10^{-15}	0.0346	0.301
sub0	0.50	0°	0.02	-2.45×10^{-3}	−0.03	-5.80×10^{-4}	0.0340	0.300
sub2	0.50	2°	0.02	-3.36×10^{-3}	+0.12	-7.23×10^{-4}	0.0337	0.295
super	0.75	2°	0.02	-1.19×10^{-2}	−8.92	-3.78×10^{-3}	0.0328	0.429
anchor	0.80	1.25°	0	-8.67×10^{-3}	−3.53	-2.35×10^{-3}	0.0335	0.320

case is done if and only if its `result.json` exists, and that file is written last and atomically. The campaign could therefore be resumed after every interruption, including 24 sleeps of the machine, most of them thermal.

Failures. Twenty cases at 256×97 and five at 512×193 failed Equation (10); three of the failures appear at both levels. Of these 25 failures, 23 had a residual that stalled below 5 orders while C_L and C_D were still moving. In the other two the residual dropped far enough, but C_L still varied by 1.6×10^{-5} and 4.0×10^{-6}. All of them sit in the transonic, higher-lift corner of the box (Figure 3). They are excluded from the dataset and listed with their reasons, not patched.

The dataset. The final dataset holds 881 training, 126 validation, 62 in-range and 15 extrapolation cases at 256×97, and 61 and 14 test cases at 512×193. Scoring uses the 61 in-range and 14 extrapolation cases that converged at both levels.

Regime. A case is *supercritical* if its maximum surface Mach number in the truth exceeds 1, and *subcritical* otherwise. The 61 in-range cases split 37 subcritical and 24 supercritical. All 14 extrapolation cases are supercritical, so no subcritical extrapolation figure exists.

Showcase cases. Six cases are picked by rule, so that nobody picks flattering ones. The surviving in-range cases are sorted by maximum surface Mach number, and the cases at sorted index round($p(n-1)$) are taken for $p = 0.10$, 0.30, 0.50, 0.70, 0.85 and 0.95. This gives three subcritical cases (`test_in-0022`, `-0021`, `-0043`) and three supercritical ones (`-0004`, `-0031`, `-0040`).

4 The learned models

4.1 POD baseline

POD needs no change of representation, because every case is the same array shape.

The basis. Stack the N standardised training fields as rows of $\boldsymbol{Y} \in \mathbb{R}^{N \times D}$, with mean $\bar{\boldsymbol{y}}$. The snapshot method [9] gives the eigenpairs $(\lambda_k, \boldsymbol{v}_k)$ of $(\boldsymbol{Y} - \bar{\boldsymbol{y}})(\boldsymbol{Y} - \bar{\boldsymbol{y}})^\mathsf{T}$, and from them the orthonormal modes

$$\boldsymbol{\phi}_k = \lambda_k^{-1/2} (\boldsymbol{Y} - \bar{\boldsymbol{y}})^\mathsf{T} \boldsymbol{v}_k. \tag{13}$$

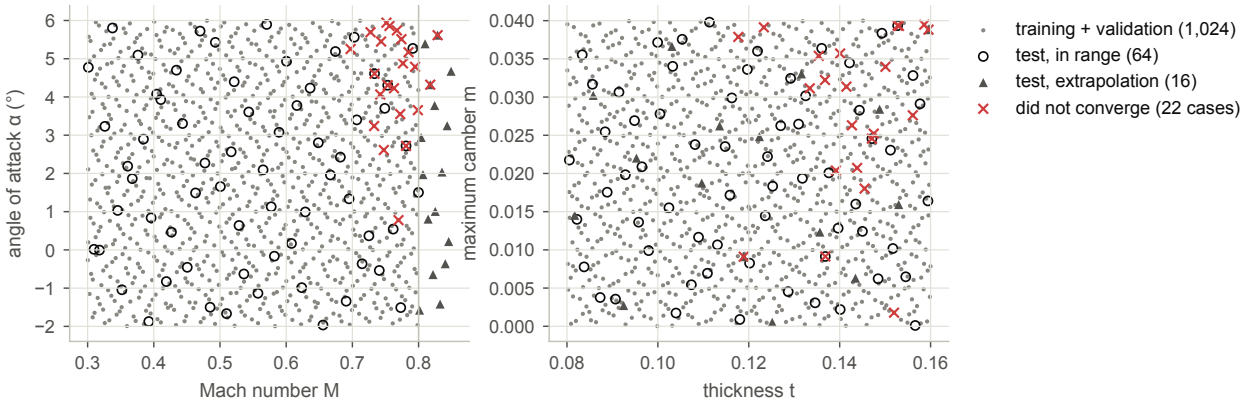


Figure 3: The case design, projected onto two pairs of inputs. Crosses mark the 22 distinct cases that failed the convergence rule at either grid level. All have $M_\infty \geq 0.697$, and 21 of the 22 have $\alpha \geq 2.6°$: shock-dominated cases on which the residual stalls while the forces still move. The vertical line marks the edge of the training range, $M_\infty = 0.80$.

The prediction. A new case with inputs $\boldsymbol{\mu} = (M_\infty, \alpha, t, m)$ is predicted as

$$\hat{\boldsymbol{y}}(\boldsymbol{\mu}) = \bar{\boldsymbol{y}} + \sum_{k=1}^{K} a_k(\boldsymbol{\mu})\, \boldsymbol{\phi}_k, \tag{14}$$

where each weight a_k is interpolated over the inputs (scaled to the unit box) by a thin-plate-spline radial basis function.

Selection. K and the RBF smoothing were chosen on the validation set. The chosen values (256 modes, smoothing 10^{-2}) lie on the edges of the search grids, so the baseline may be under-tuned.

4.2 Fourier neural operator

Appendix B explains the network in detail, with its tensor shapes and parameter counts. The FNO works in the O-grid's index space. Around the airfoil the grid is periodic, so a Fourier transform along i is exact and a learned mode means the same physical wavenumber at every resolution.

Inputs and outputs. The FNO takes six channels per node:

- the radially compressed position $(x - \frac{1}{2},\ y)\ \ln(1+r)/r$, where r is the distance from mid-chord;
- the ring index $j/(N_J - 1)$;
- M_∞, $\cos\alpha$ and $\sin\alpha$, the same at every node.

The shape therefore reaches the model only through the node positions.

It predicts four channels, each of order one across the Mach range:

$$\boldsymbol{y} = \Big(\hat{u} - \cos\alpha,\quad \hat{v} - \sin\alpha,\quad C_p,\quad \frac{\hat{\rho} - 1}{\frac{1}{2}M_\infty^2}\Big). \tag{15}$$

Each channel is then standardised over the training set. Without this scaling, $\hat{p} - 1$ would shrink like M_∞^2 and low-Mach cases would barely register in the loss.

Table 4: FNO tuning ($n = 896$, loss A, 4 layers, 100 epochs, on the Apple GPU), and the final runs of the selected configuration (300 epochs). Validation is relative L2 on standardised targets.

Tuning					Final runs (width 48, modes 24×8)				
Tag	width	modes	best val	wall s	Run	loss	best val	epoch	wall s
tune-0	32	16×8	0.0315	419	n 896, seed 0	A	0.01261	278	2,160
tune-1	32	16×12	0.0308	443	n 896, seed 1	A	0.01274	293	2,242
tune-2	32	24×8	0.0310	443	n 896, seed 2	A	0.01178	294	2,237
tune-3	32	24×12	0.0310	488	n 896, seed 0	B	0.4617	50	605
tune-4	48	16×8	0.0229	658	n 64	A	0.0849	297	257
tune-5	48	16×12	0.0223	763	n 128	A	0.0507	288	421
tune-6	48	24×8	**0.0221**	793	n 256	A	0.0332	284	737
tune-7	48	24×12	0.0235	859	n 512	A	0.0178	274	1,384

Architecture. A pointwise layer lifts the six inputs to w channels. Four Fourier layers follow, each

$$\boldsymbol{h} \leftarrow \sigma\Big(\mathcal{F}^{-1}[\boldsymbol{R} \cdot (\mathcal{F}\boldsymbol{h})_{|k_i|<k_I,\ k_j<k_J}] + \boldsymbol{W}\boldsymbol{h}\Big), \tag{16}$$

with learned complex weights $\boldsymbol{R}$ on the retained modes, a pointwise linear path $\boldsymbol{W}$ and GELU activations σ (none after the last layer). A projection $w \to 128 \to 4$ gives the output.

Radial padding. Radially the field is not periodic, so it is zero-padded by

$$P(N_J) = \mathrm{round}\big(3(N_J - 1)/16\big) - 1 \tag{17}$$

rows before the transform: 8 rows at 128×49 and 17 at 256×97. The discrete Fourier period, $(N_J + P)/(N_J - 1) = 19/16$ in η, is then the same at both resolutions. An earlier padding of $(N_J - 1)/6$ matched the padded span instead of the period, and one spectral layer then differed by 4–71 % between the two resolutions.

Loss and training. Loss A is the per-sample relative L2 error of each output channel, averaged over channels; loss B adds the same term on the wall row, weighted 10:

$$\mathcal{L}_A = \frac{1}{BC} \sum_{b,c} \frac{\|\hat{\boldsymbol{y}}_{bc} - \boldsymbol{y}_{bc}\|_2}{\|\boldsymbol{y}_{bc}\|_2 + \epsilon}, \qquad \mathcal{L}_B = \mathcal{L}_A + 10\, \mathcal{L}_A\big|_{j=0}. \tag{18}$$

Training used AdamW [7] with learning rate 10^{-3}, cosine decay, weight decay 10^{-4} and batch 8, for at most 300 epochs with early stopping at a patience of 30. It ran on the 128×49 subgrid (every second node of the 256×97 fields), and the model is evaluated at 256×97. That resolution transfer is scored separately.

Tuning. Eight configurations were trained for 100 epochs and selected on validation loss (Table 4). Width separates them; the mode counts move the loss by about as much as a change of seed would.

4.3 PINN

The PINN solves one test case at a time, on the same domain as SU2. It is given that case's inputs and the O-grid map, and never any of SU2's field values. Two formulations were built. The first, on the Euler equations, failed in a way worth recording. The second, on the full-potential equation, is the one scored. Appendix C explains both in detail.

4.3.1 The Euler PINN, and why it converges to fluid at rest

Formulation. A fully connected tanh network takes $(\cos 2\pi\xi,\ \sin 2\pi\xi,\ \eta)$, which makes it periodic around the airfoil. It outputs $\ln\rho$ and a raw velocity $\tilde{\boldsymbol{u}}$, and two constraints are built in:

- *Wall tangency*, exact by construction: $\boldsymbol{u} = \tilde{\boldsymbol{u}} - (1-\eta)^4(\tilde{\boldsymbol{u}}\cdot\hat{\boldsymbol{n}}_w)\hat{\boldsymbol{n}}_w$.
- *Constant total enthalpy*, so the pressure is algebraic, $p = \frac{\gamma-1}{\gamma}\rho(H_\infty - \frac{1}{2}|\boldsymbol{u}|^2)$ with $H_\infty = 1/((\gamma-1)M_\infty^2) + \frac{1}{2}$ (in $\rho_\infty U_\infty^2$ units).

The residuals of continuity and the two momentum equations are written in strong conservation form in grid coordinates,

$$\mathcal{R} = \frac{1}{J}\left[\frac{\partial}{\partial\xi}(y_\eta F - x_\eta G) + \frac{\partial}{\partial\eta}(-y_\xi F + x_\xi G)\right], \qquad J = x_\xi y_\eta - x_\eta y_\xi, \tag{19}$$

with the metric terms from automatic differentiation of Equation (8). The free stream is a soft penalty on the outer circle, and the Kutta condition a soft penalty on the velocity at the trailing edge.

The failure. The warm-up is a circular cylinder at $M_\infty = 0.1$, with the circulation held at zero. It must reproduce $C_p = 1 - 4\sin^2\theta$ to within 0.1. It failed with a largest error of 3.94: the network had converged to *fluid at rest* around the body. The wall speed was 0.004 where potential flow gives 2.0, and $C_p = 0.937$ everywhere (Figure 4).

Refuted hypotheses. Five hypotheses were each refuted by a probe:

- weighting the residual by J^p for $p = 0$, 0.5 and 1;
- an exact reparameterisation against low-Mach stiffness;
- a 10-chord outer boundary;
- a homentropic closure;
- the real showcase case in place of the cylinder.

The decisive probe. A network was fitted, by supervision, to the analytic potential flow. It would pass the warm-up (shoulder speed 1.995, $\max|\Delta C_p| = 0.030$). Yet its PINN loss is 0.31, about 1,000 times the rest state's $\sim 10^{-4}$, because small fitting errors are large derivative errors. A further 1,000 PINN-only steps from that fit drive it away from the answer.

Root cause. Fluid at rest near the body satisfies the strong-form equations and the wall condition exactly; it violates only the weakly enforced far field. Every nearly correct flow costs more, so descent leaves even the true solution. Within the configuration tested here the fault was in the formulation, not in a parameter. It is not a property of Euler PINNs in general: Wassing, Langer and Bekemeyer [11] train an Euler PINN with a soft far field to within 1 % of a finite-volume reference on transonic NACA 0012 cases (Section 7).

4.3.2 The full-potential PINN

Formulation. For the full-potential equation in subcritical flow, the residual loss is (nearly) convex in the potential, with no rest attractor, and a Dirichlet far field makes rest inadmissible.

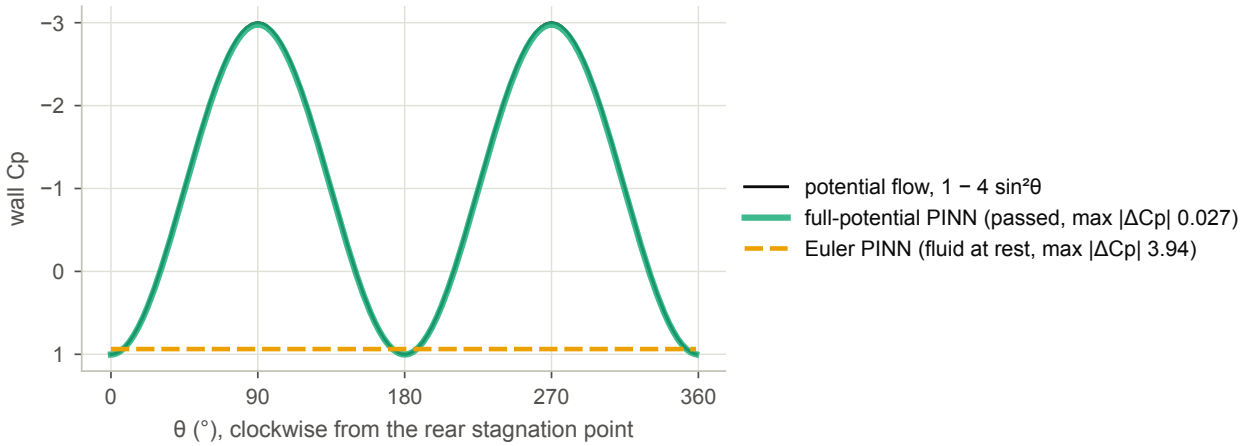


Figure 4: The cylinder warm-up at $M_\infty = 0.1$ (pass: $\max|\Delta C_p| \le 0.1$). The Euler PINN reaches $C_p \approx 0.94$ all round the wall, the stagnation value: fluid at rest. The full-potential PINN lies on the analytic curve (largest error 0.027), with C_L and C_D of -4×10^{-4} and -1×10^{-3}.

One scalar network $\psi(\xi,\eta)$ and a learned circulation Γ give the velocity potential, in units of $U_\infty c$:

$$\phi = x\cos\alpha + y\sin\alpha + \Gamma\,\Theta_v(x,y) + \psi(\xi,\eta)\,(1-\eta), \qquad \Theta_v = 1 - \frac{\operatorname{atan2}(y,\ x-\frac{1}{4}) \bmod 2\pi}{2\pi}. \tag{20}$$

- The factor $(1-\eta)$ vanishes on the 100-chord circle, so the far field is exact (the neglected doublet is $\sim 10^{-3}$ there).
- Θ_v is the angle of a clockwise point vortex at the quarter chord. Its unit jump lies on the ray $y=0$, $x>\frac{1}{4}$, which in the domain is exactly the grid line $\xi=0$, the wake cut. Its gradient, $1/(2\pi r)$, is continuous across the cut and regular at the trailing edge.

The state. The state follows algebraically. The velocity is $\boldsymbol{u} = \nabla\phi$ through the metric terms,

$$u = \frac{\phi_\xi y_\eta - \phi_\eta y_\xi}{J}, \qquad v = \frac{-\phi_\xi x_\eta + \phi_\eta x_\xi}{J}, \tag{21}$$

and the density and pressure come from isentropic Bernoulli at the free-stream total enthalpy:

$$\hat\rho = \left[1 + \tfrac{\gamma-1}{2}M_\infty^2\left(1-|\boldsymbol{u}|^2\right)\right]^{1/(\gamma-1)}, \qquad \hat p = \hat\rho^{\,\gamma}. \tag{22}$$

The equation. The only equation left is steady continuity,

$$\nabla\cdot(\rho\nabla\phi) = 0, \tag{23}$$

written in the strong conservation form Equation (19) with $(F,G) = \rho(u,v)$. Momentum holds by construction, because the flow is irrotational, isentropic and at constant H. Equation (23) is exact for shock-free inviscid flow and only an *isentropic approximation* at shocks. On supercritical cases it therefore solves a different equation from SU2's: there is no entropy jump, so its shocks are those of isentropic flow.

Loss. The loss has three terms:

$$\mathcal{L} = \underbrace{\frac{1}{N_r}\sum_n \mathcal{R}_n^2}_{\text{continuity}} + w_{\text{wall}}\underbrace{\frac{1}{N_w}\sum_n (\boldsymbol{u}\cdot\hat{\boldsymbol{n}}_w)_n^2}_{\partial\phi/\partial n=0} + w_K\underbrace{\left(|\boldsymbol{u}|^2_{(\delta,0)} - |\boldsymbol{u}|^2_{(1-\delta,0)}\right)^2}_{\text{equal-speed Kutta}}, \qquad w_{\text{wall}} = w_K = 10. \tag{24}$$

Two trailing-edge fixes. Both fixes were found on the first showcase case. There, the initial residual was $\sim 10^5$, with 99.9 % of it carried by the 10 collocation points nearest the trailing edge.

1. *The circulation term.* It was first $\Gamma\xi$, and $|\nabla\xi| \sim r^{-1/2}$ at the trailing edge, where $x_\xi = y_\xi = 0$. With Θ_v the circulation-only residual fell from 1.2×10^4 to 8×10^{-3}.
2. *The residual near the edge.* The network term's second derivatives are singular at the sharp corner, as the true corner flow is. Collocation points within 0.02 chord of the trailing edge are therefore dropped from the residual: 197–225 of 20,000 at each resample.

The Kutta condition. The first Kutta form, $(\partial\phi/\partial\xi)^2$ at the trailing-edge point, was degenerate. There $x_\xi = y_\xi = 0$, so the free-stream and circulation terms contribute nothing to it, and it could not constrain Γ. The equal-speed form in Equation (24) replaces it. It compares $|\boldsymbol{u}|^2$ on the two surfaces at $\xi = \delta$ and $1 - \delta$, with $\delta = 0.055$, about 0.03 chord upstream of the edge.

Training.

- *Network*: 7 hidden layers of 96 tanh units.
- *Collocation*: 20,000 points uniform in (ξ, η), which the map clusters near the wall, redrawn every 1,000 steps; 512 wall points.
- *Adam* [4]: 30,000 steps in float32 on the Apple GPU, with the learning rate decaying exponentially from 10^{-3} to 10^{-4}.
- *L-BFGS* [6]: up to 5,000 iterations in float64 on the CPU, which has the float64 that the Apple GPU lacks.

One variant balances the weights by gradient norms [12], every 100 steps:

$$\lambda_k \leftarrow (1 - a)\lambda_k + a\,\frac{\max_\theta |\nabla_\theta \mathcal{L}_{\text{res}}|}{\overline{|\nabla_\theta \mathcal{L}_k|}}, \qquad a = 0.9. \tag{25}$$

The runs. Before S3 started, every planned run was timed for a few steps. The stage refuses to start if the projected total exceeds its budget.

Eight runs were made (Table 5):

- the six showcase cases with fixed weights (`pure`);
- gradient-norm balancing on the median-Mach case (`gradnorm`);
- one run without the Kutta term (`nokutta`), for prediction P3.

The specification also called for runs with an artificial viscosity at shocks, $\nu = 10^{-3}$ and 3×10^{-4}. They were dropped by the author's decision of 5 October 2026: at about a day each, the four of them would have added roughly 46 hours on two shards.

5 Scoring

Truth and the floor. The truth is SU2 on the 512×193 grid, read at every second node. Because the grids nest, those nodes are exactly the 256×97 nodes, and every contestant is evaluated on them. The floor is SU2's 256×97 solution scored against that truth. It is the error that being on the evaluation grid already costs, so a surrogate at the floor is as good as SU2 on that grid.

Table 5: The eight PINN runs (stage S3). Two runs shared the machine at a time, which inflates the mean wall time by at most about 1.2 times. Losses are the final values after L-BFGS.

Variant	Case (regime)	Wall h	CPU h	Residual	Wall	Kutta	Total
pure	0022 (sub)	5.45	9.74	2.25×10^{-3}	1.3×10^{-4}	1.2×10^{-7}	3.58×10^{-3}
pure	0021 (sub)	5.33	9.27	4.91×10^{-3}	3.9×10^{-4}	5.9×10^{-6}	8.92×10^{-3}
pure	0043 (sub)	4.97	9.23	1.10×10^{-3}	4.9×10^{-5}	8.8×10^{-8}	1.59×10^{-3}
pure	0004 (super)	5.01	9.40	2.98×10^{-3}	2.2×10^{-4}	1.2×10^{-7}	5.15×10^{-3}
pure	0031 (super)	6.47	11.57	5.31×10^{-3}	4.2×10^{-4}	4.5×10^{-7}	9.50×10^{-3}
pure	0040 (super)	6.44	11.56	2.38×10^{-2}	2.1×10^{-3}	5.7×10^{-6}	4.51×10^{-2}
nokutta	0022 (sub)	5.40	10.08	1.82×10^{-3}	7.8×10^{-5}	0	2.61×10^{-3}
gradnorm	0004 (super)	4.96	7.20	9.11	9.4×10^{-3}	1.9×10^{-13}	82.2

One force integrator. The integration is vertex-lumped: each wall node carries half of each adjacent edge's outward normal, $\boldsymbol{n}_i = \frac{1}{2}(\boldsymbol{n}_{e,i-1} + \boldsymbol{n}_{e,i})$. With $\boldsymbol{f} = -\sum_i C_{p,i}\, \boldsymbol{n}_i$, the coefficients are

$$C_D = f_x \cos\alpha + f_y \sin\alpha, \qquad C_L = -f_x \sin\alpha + f_y \cos\alpha, \qquad C_M = \sum_i \big[(x_i - \tfrac{1}{4}) f_{y,i} - y_i f_{x,i}\big]. \tag{26}$$

On SU2's own solutions this reproduces SU2's reported coefficients to 5×10^{-11}. The edge-midpoint trapezoid matched C_L and C_D but missed C_M by 1.6×10^{-5}, which fails the 10^{-6} check.

Errors. Force errors are absolute, with C_D in counts, because percentages of near-zero inviscid drag mean nothing. The wall-pressure error is the RMS of ΔC_p over the wall nodes. The near-field error is the area-weighted relative L2 error over the nodes within 2.5 chords of mid-chord:

$$e_{\mathrm{nf}} = \left(\frac{\sum_i w_i \, |\hat{\boldsymbol{q}}_i - \boldsymbol{q}_i|^2}{\sum_i w_i \, |\boldsymbol{q}_i - \boldsymbol{q}_\infty|^2} \right)^{1/2}, \qquad |\boldsymbol{x}_i - (\tfrac{1}{2}, 0)| \le 2.5, \tag{27}$$

where w_i is the dual-cell area.

Shocks. On each surface that goes supersonic, the shock is the steepest pressure rise downstream of the suction peak, among segments whose upstream node is supersonic. Without that restriction the rule finds the trailing-edge recompression on every real solution. The overshoot is the largest $|\Delta C_p|$ within 0.05 chord of the true shock.

Physics checks. These run on every column, SU2 included:

- the entropy in subcritical cases;
- $|H/H_\infty - 1|$;
- the RMS velocity through the wall;
- the net mass flux through the closed grid lines $j = 8$, 24, 48 and 72.

A check that a contestant satisfies by construction is labelled as such. For the potential PINN, entropy and enthalpy are identities, and wall tangency is a result.

Table 6: The predictions recorded before any model ran, and their outcomes.

Prediction	Outcome
P1. For a single new case, SU2 beats the PINN on cost by one to two orders of magnitude.	**Held on wall time only** (88.5×, 1.95 orders). **Not held** on CPU time or per phase (174–220×, 2.24–2.34 orders): SU2 wins by more than predicted.
P2. The FNO beats "just run SU2" only after about as many new cases as trained it (∼1,000).	**Held.** Break-even at $n^* = 1{,}126$ new cases. The data-efficiency curve is still 3.4× the floor at $n = 896$, so the "unless" clause does not fire.
P3. Without the Kutta condition the PINN's residuals look acceptable while its lift is wrong.	**Partly.** Without the condition the residual is as small as with it (1.8×10^{-3} against 2.3×10^{-3}) and the lift is 84 % low. But with the condition it is still 77 % low: the Kutta term made no material difference to the circulation.

Cost. Each method's cost is a fixed part plus a part per new case,

$$C_m(n) = C_m^{\text{fixed}} + n\, c_m, \qquad n_m^* = \min\{n : C_m(n) < n\, c_{\text{SU2}}\}. \tag{28}$$

- SU2's cost per case is its median CPU time on the test cases.
- The fixed cost of the FNO and of POD is the whole campaign, every validation case included, plus model search and final training.
- GPU work is charged at wall time, because process CPU time cannot see it; CPU work is charged at CPU time. The PINN is charged per phase: Adam (GPU) at wall time, L-BFGS (CPU) at CPU time.

6 Results

6.1 The three predictions

Table 6 gives the verdicts. The costs behind P1 are in Table 7.

P1 depends on the cost basis. The mean `pure` run took 20,201 s of wall time and 36,464 s of CPU time, because L-BFGS ran multithreaded on the CPU. SU2 took a median 228.1 s of wall time and 209.9 s of CPU time per case.

- Wall time against wall time gives 88.5×, inside the predicted band.
- The per-phase rule, which the scorer applies to every other method, gives 174–220×, outside it.

The per-phase cost is a bracket, not a single figure, because the S3 runs did not record the Adam phase's own CPU time. A later measurement of that fraction (0.346) gives 204×.

6.2 Accuracy by regime and set

Table 8 and Figure 5 give the main result, with the floor beside every error.

What the table says:

Table 7: Costs, in seconds. SU2 per case is the median over the 80 test cases at 256×97. The FNO and POD fixed costs include the full campaign (229,211 s of CPU time). The PINN has no fixed cost.

Method	Fixed	Per case (wall)	Per case (CPU)	Basis used for break-even
SU2 256×97	0	228.1	209.9	CPU, 209.9
FNO	236,237	0.054 per query		campaign + 8 tuning runs (4,867) + training (2,160)
POD	229,562	0.007 per query		campaign + mode and smoothing search
PINN (`pure`, mean of 6)	0	20,201	36,464	per phase: 36,464–46,122

Table 8: Errors by set and regime, with the floor first. Near field: mean of Equation (27); C_L and C_D: largest absolute error in the group; C_p: mean RMS wall error. FNO rows are seed 0, $n = 896$, loss A, scored on its 256×97 evaluation grid and on its 128×49 training grid (against the truth at every fourth fine node). PINN rows cover the three showcase cases of each regime.

Set / regime	Method	Near field	C_L max	C_D max (counts)	C_p RMS
in range subcritical (37)	**floor**	**0.0080**	**0.0073**	**6.9**	**0.0357**
	FNO 256×97	0.0423	0.0312	31.3	0.0358
	FNO 128×49	0.0102	0.0095	7.5	0.0334
	POD	0.0168	0.0097	14.2	0.0371
	PINN (3)	0.5155	0.2461	41.1	0.1603
in range supercritical (24)	**floor**	**0.0241**	**0.0140**	**17.2**	**0.0341**
	FNO 256×97	0.0575	0.0431	40.4	0.0521
	FNO 128×49	0.0374	0.0314	46.6	0.0470
	POD	0.1462	0.0750	122.2	0.1190
	PINN (3)	0.6273	1.4028	776.5	0.4045
extrapolation supercritical (14)	**floor**	**0.0651**	**0.0247**	**32.4**	**0.0598**
	FNO 256×97	0.2318	0.1847	180.6	0.2038
	FNO 128×49	0.2320	0.1876	178.3	0.1966
	POD	0.5151	0.3483	530.2	0.3512

- **In range and subcritical, the FNO at its own resolution is near the floor**: 1.3 times on the near field, 1.3 times on C_L and 1.1 times on C_D. Evaluated at 256×97 it is 5.3 times the floor, and POD beats it there on the near field, C_L and C_D. The resolution transfer costs the FNO more than any other single factor in range.
- **In range and supercritical, the FNO beats POD by 2.5 times** on the near field and 3 times on C_D, at 2.4 times the floor. Shocks that move with the inputs are exactly where a fixed linear basis struggles.
- **Extrapolation is where every surrogate fails.** The FNO is 3.6 times the floor, and its C_D error reaches 181 counts, biased rather than scattered (median -78 counts); POD is 7.9 times the floor. Here the FNO is equally wrong at both resolutions.
- **The PINN is not competitive in any column**: 65 and 26 times the floor on the near field.

6.3 Fields and wall pressure

Figure 6 shows the Mach field of one subcritical and one supercritical case from each method.

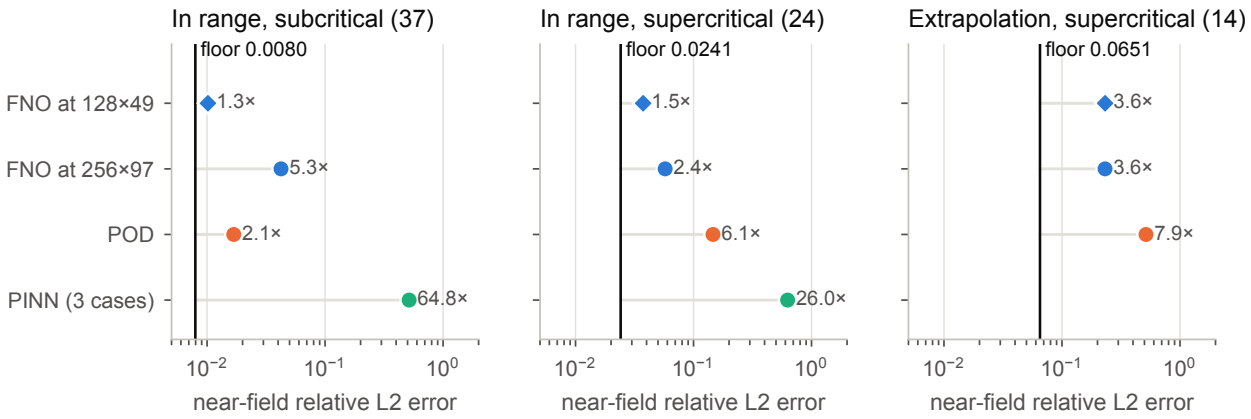


Figure 5: Near-field error against the floor (vertical line); each label is the ratio to the floor. The FNO at 128×49 is scored on its own training grid, so its comparison grid is coarser than the floor's.

- On the subcritical case the FNO and POD are close to the truth at this scale. The PINN's field is nearly symmetric about the chord: the faster flow over the upper surface, which carries the lift, is missing.
- On the strongest supercritical showcase case (`test_in-0040`, maximum surface Mach 1.61) the FNO places the supersonic pocket and its terminating shock well. POD shrinks the pocket and moves the shock forward. The PINN has no supersonic region at all: its lift on this case is -0.015, against SU2's 1.388.

Figure 7 gives the wall pressure on all six showcase cases.

6.4 Lift

The PINN's lift error is the clearest failure (Figure 8, Table 9). Five of the six `pure` runs underpredict lift, three of them to roughly zero. By Kutta–Joukowski, $L' = \rho_\infty U_\infty \Gamma$, so $\Gamma = C_L/2$ in units of $U_\infty c$; a lift 4.4 times too small on `test_in-0022` is a circulation 4.4 times too small.

The Kutta term can pin Γ. The equal-speed term does pin the circulation of a true potential flow. On the exact incompressible flat-plate (Joukowski) flow at $\alpha = 2.19°$, evaluated 0.03 chord upstream of the trailing edge, the quantity $|\boldsymbol{u}|^2_{\text{upper}} - |\boldsymbol{u}|^2_{\text{lower}}$ is:

- -0.421 at $\Gamma = 0$;
- -0.277 at $\Gamma = 0.0387$, the value recovered for the `pure` run;
- $+0.027$ at the Kutta value $\Gamma = \pi \sin\alpha = 0.120$.

So the network is not a potential flow near the edge. A wrong-Γ potential flow would pay about 100 times the penalty of the right one. A network that meets the term to 10^{-7} at the wrong Γ is therefore not a potential-flow solution near the trailing edge. Whether the network's representation, the 0.02-chord exclusion or both is responsible is not isolated here.

6.5 PINN training

The loss does not reveal the circulation. Figure 9(a) shows that the residual loss converges equally well with and without the Kutta term.

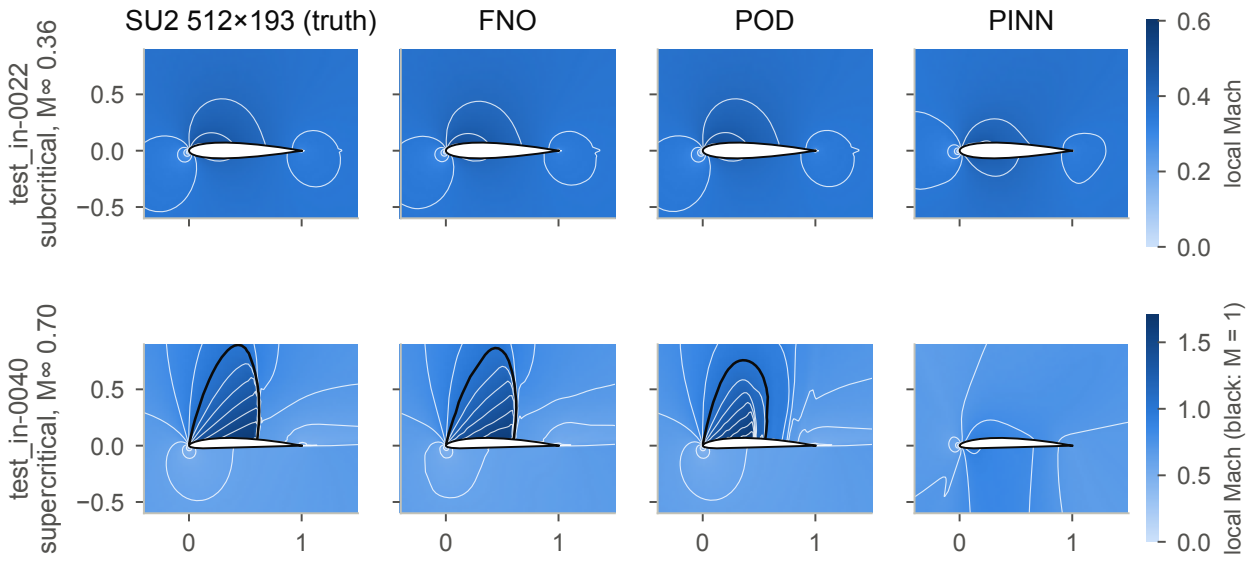


Figure 6: Local Mach number near the section: the truth and each contestant. Thin white lines are iso-Mach contours (every 0.05 in the top row, 0.1 in the bottom); the black line is the sonic line. Each row has its own colour scale.

Table 9: The PINN per case (test_in, 256×97). The floor's C_L errors on these cases are 0.0003–0.0086.

Run	Regime	SU2 C_L	PINN C_L	C_L error	C_D error (counts)	Near field
pure / 0022	sub	0.3192	0.0732	−0.2461	+41.1	0.559
nokutta / 0022	sub	0.3192	0.0526	−0.2667	+30.0	0.569
pure / 0021	sub	0.0922	−0.0012	−0.0934	−5.6	0.629
pure / 0043	sub	0.2799	0.1008	−0.1790	+15.1	0.359
pure / 0004	super	1.0139	1.0662	+0.0523	+88.6	0.218
pure / 0031	super	0.2949	0.0007	−0.2942	−9.9	0.584
pure / 0040	super	1.3879	−0.0149	−1.4028	−776.5	1.080
gradnorm / 0004	super	1.0139	−0.3272	−1.3411	+724.8	1.273

The gradnorm run. Gradient-norm balancing failed in a specific way (Figure 9(b)):

- From step 500 on, the weighted wall term was at least 96 % of the total at every logged step, while the balanced wall weight ranged from 692 to 1.3×10^5.
- The wall loss itself stayed at 0.03–0.25, 33 times the fixed-weight run's at the end of Adam.
- L-BFGS then minimised an objective that was 99.99 % wall term. It cut the wall loss tenfold and made the equation residual 380 times worse, from 0.024 to 9.11.

The run's lift has the wrong sign. The Kutta weight also ran away, to a logged peak of 7×10^{10}, but it multiplied a Kutta loss below 10^{-9} and barely moved the total.

6.6 FNO: seeds, loss B and shocks

Seeds. Across the three seeds at $n = 896$, the near field varies by a few per cent of its value. The extrapolation C_L maximum, however, varies by 40 % (0.173–0.243), so a single-seed extrapolation lift figure is not reliable to better than that.

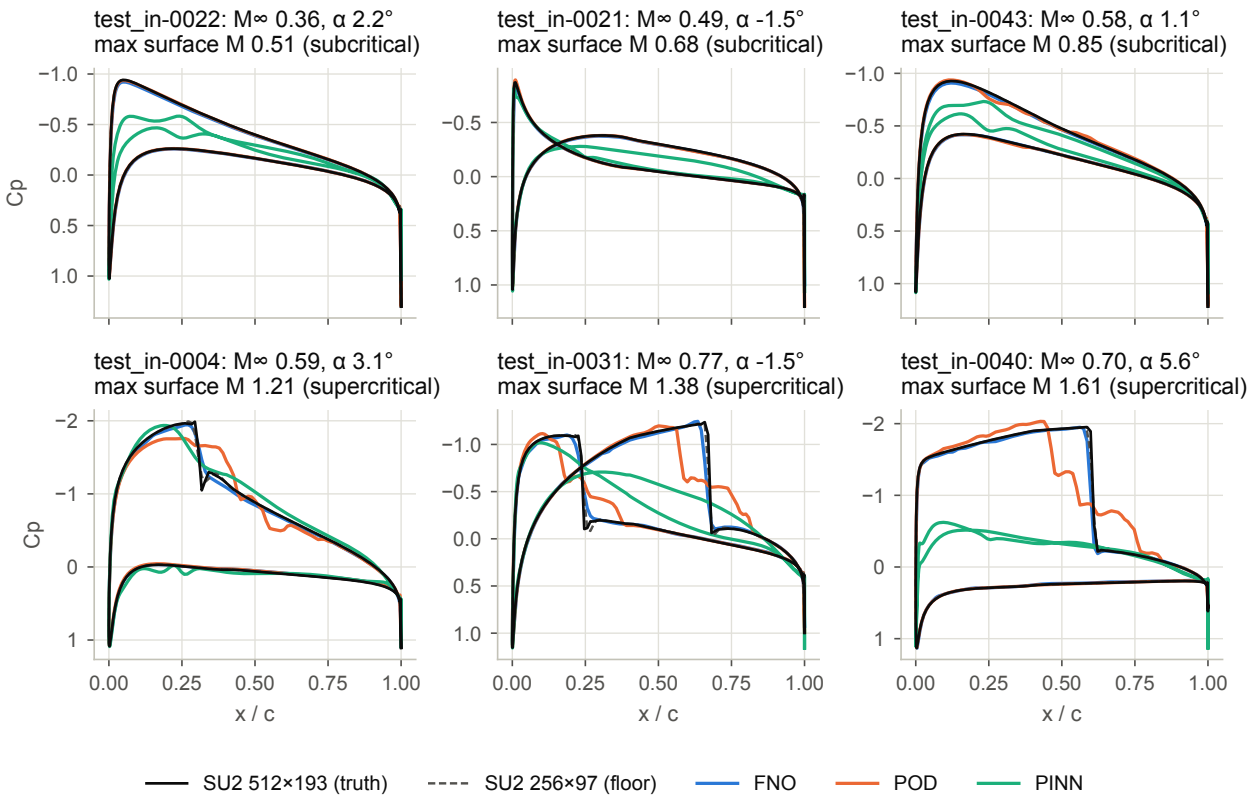


Figure 7: Wall pressure on the six showcase cases, in order of maximum surface Mach number (C_p axis inverted). The floor lies under the truth everywhere. The FNO follows the truth closely and smears the shocks a little. POD's shocks are displaced and broken into steps. The PINN's suction peaks are too weak, the signature of a circulation far below the Kutta value.

Loss B. Loss B is worse than loss A on the near field and C_D in every set and regime, at both grids. It is worse on C_L in every cell but one: in-range subcritical at 256×97, where it is 23 % better. It is worst on supercritical lift (C_L maximum 0.249 against 0.043). It stopped early, at epoch 81, so this compares the runs as made, not converged runs.

Shocks. Table 10 gives the shock scores.

- In range, the FNO places shocks at the floor's median accuracy but misses 1–2 of 27 and smears them, with twice the floor's median overshoot.
- POD never misses a shock but places them 0.06–0.11 chord off at the median.
- Out of range, the FNO's position errors reach 0.21 chord.

6.7 Data efficiency and break-even

Figure 10 and Table 11 show the test error against the number of SU2 cases paid for. Both models are still improving at $n = 896$, and neither approaches the floor. A smaller training set would break even sooner, at about 199 cases for $n = 64$, but its answers would be 7.6 times the floor rather than 3.4. Figure 11 draws Equation (28): the FNO and POD lines cross SU2's at 1,126 and 1,094 new cases.

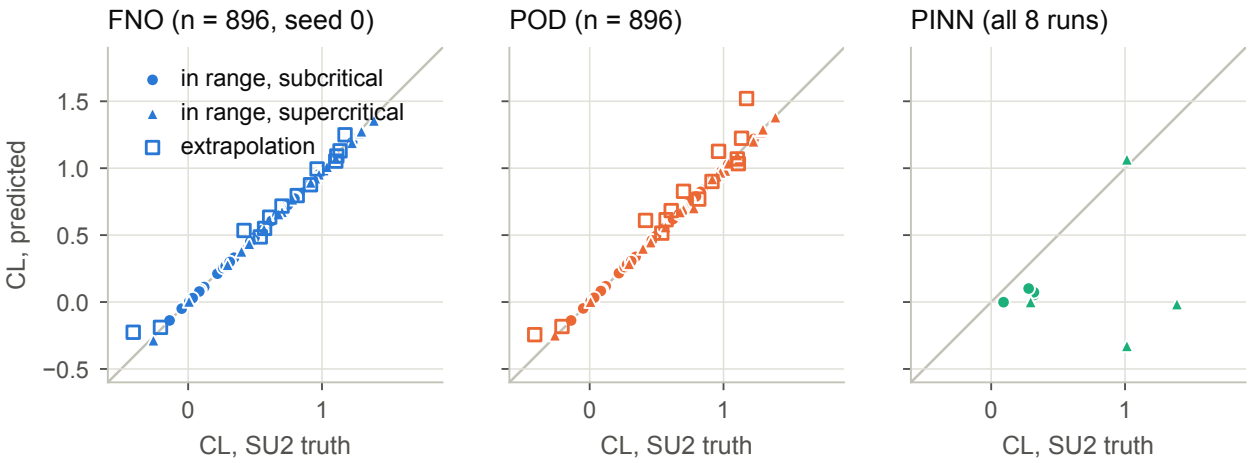


Figure 8: Predicted against true C_L at 256×97. Filled markers: in range; hollow: extrapolation. In extrapolation the FNO's lift errors reach +0.18 and POD's +0.35, and POD over-predicts at the median (+0.06). The PINN's points lie well below it, except `pure` on `test_in-0004`. The `gradnorm` run on that case is at (1.01, −0.33).

Table 10: Shocks: 27 true shocks on the 24 in-range supercritical cases and 21 on the 14 extrapolation cases. Position error in chords; overshoot is the largest $|\Delta C_p|$ within 0.05 chord of the true shock.

Set	Method	Missed	$\|\Delta x\|$ median / max	Overshoot median / max
in range	**floor**	**0**	**0 / 0.019**	**0.19 / 0.54**
	FNO 256×97	2	0 / 0.058	0.42 / 1.05
	FNO 128×49	1	0 / 0.069	0.29 / 0.92
	POD	0	0.061 / 0.135	0.65 / 1.34
	FNO loss B	2	0.012 / 0.068	0.59 / 1.29
	PINN `pure` (3 cases, 4 shocks)	1	0.046 / 0.181	0.74 / 1.67
extrapolation	**floor**	**0**	**0.008 / 0.012**	**0.69 / 1.00**
	FNO 256×97	0	0.042 / 0.213	1.13 / 1.58
	POD	0	0.106 / 0.250	1.28 / 1.60

6.8 Physics checks and failures

SU2 itself. SU2 is not free of numerical artefacts:

- *Entropy.* In subcritical cases, where it should be zero, the truth's near-field maximum entropy has median 0.020, and the floor's 0.033, so SU2 generates spurious entropy, and more of it on the coarser grid. Its area-weighted mean is small and non-negative on every case at both grids.
- *Total enthalpy* is constant to 1.2×10^{-5}.
- *No* non-positive density or pressure appears anywhere.

Failures, stated as results.

- *Non-physical states in extrapolation.* POD predicts non-positive density or pressure on 2 to 7 of the 14 extrapolation cases, and more as the training set grows. The FNO does so on 4–5 cases at $n \leq 128$ and on none at $n \geq 256$.
- *Entropy decrease.* The subcritical near-field mean entropy falls below free stream on 9–24

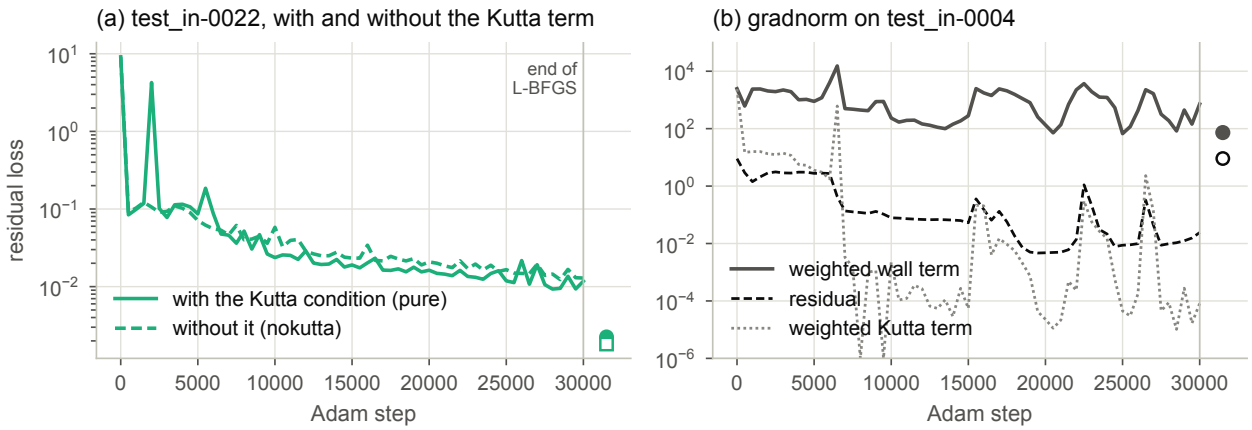


Figure 9: PINN training histories (logged every 500 Adam steps; markers right of the line: after L-BFGS). (a) With and without the Kutta term on the same case: indistinguishable losses, both with lift far too low. (b) The `gradnorm` run: the balanced wall term dominates the objective, and L-BFGS trades the residual for it.

Table 11: Data efficiency: mean near-field error over the 61 in-range test cases at 256×97 (floor 0.0143), with the SU2 cost paid for each training set (every validation case included).

n	FNO (seed 0)	FNO / floor	POD	POD / floor	Campaign, SU2-case equivalents
64	0.1088	7.6×	0.1379	9.6×	197
128	0.0858	6.0×	0.1044	7.3×	257
256	0.0698	4.9×	0.0883	6.2×	380
512	0.0561	3.9×	0.0753	5.3×	690
896	0.0483	3.4×	0.0677	4.7×	1,092

of 37 cases for the FNO and 10–23 for POD, against none for SU2. Whether any surrogate predicts an expansion shock is not measured: the scorer has no test for an entropy decrease across a compression.

- *PINN wall tangency and mass.* The potential PINN enforces both only softly: its RMS wall-normal velocity has median 0.011 (subcritical) and 0.020 (supercritical), against SU2's 10^{-8}. Its mass flux through the closed lines is one to two orders above the FNO's.
- *Not measured*: FNO ringing (only the plots show it), and the viscous PINN (dropped).

7 Discussion

The operator. The FNO does what operators promise: it learns the family well enough to beat a strong linear baseline exactly where that baseline is weak, at moving shocks. But it reaches the floor only on its own training grid and only in range. Its resolution transfer is $O(h)$, not exact: the padded field jumps to zero at the wrap, and that jump aliases differently at each resolution. In extrapolation its errors are biased in a way no in-range score would reveal. Economically it repays its training only after about as many new cases as the campaign that trained it, as predicted.

The PINN. The PINN's difficulties come from the formulation more than from the optimiser.

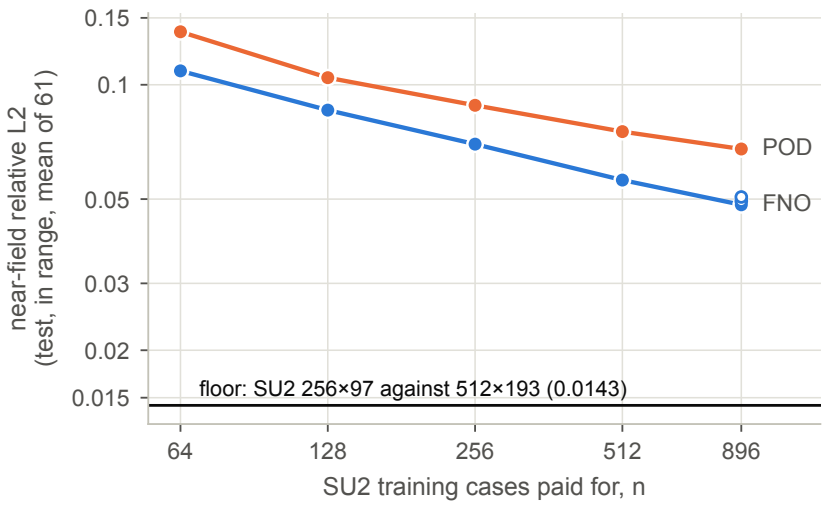


Figure 10: Data efficiency. Open markers at $n = 896$ are the FNO's seeds 1 and 2.

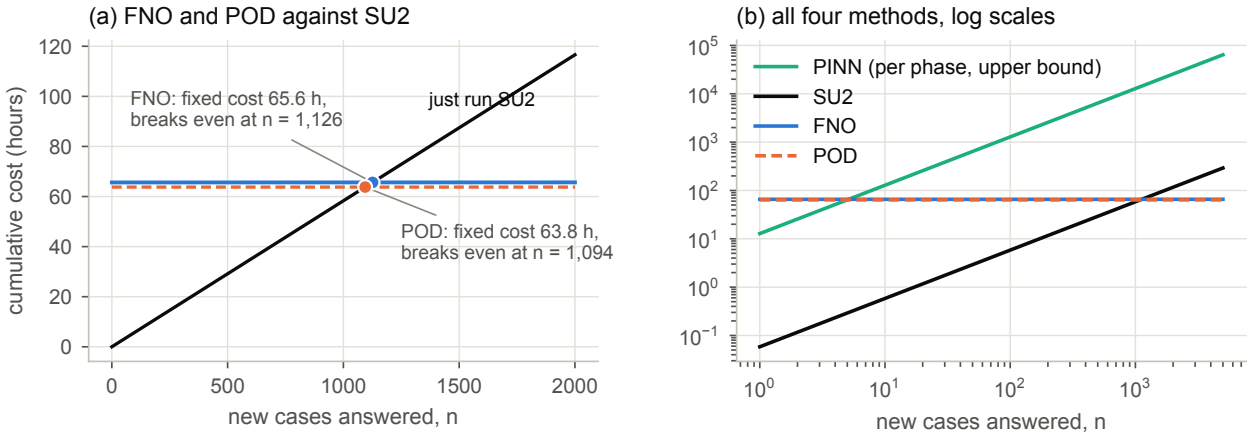


Figure 11: Cumulative cost of answering n new cases. (a) The FNO and POD each pay roughly a whole campaign up front (65.6 and 63.8 hours) and break even after about as many new cases as the campaign held. (b) The PINN, at 12.8 hours per case (per-phase upper bound), is above SU2 at every n, above POD from the fifth case and above the FNO from the sixth.

- The Euler PINN, as configured here, has a degenerate minimiser: fluid at rest, which its soft far-field condition did not exclude.
- The full-potential PINN removes that minimiser and passes the cylinder warm-up. It still under-predicts the circulation on lifting sections, while the equal-speed Kutta term is met at the wrong circulation.
- The sharp trailing edge is the crux: there the map is degenerate, the corner flow is singular, and the condition that fixes the lift lives.
- Gradient-norm balancing, a standard remedy, made one run worse by handing the objective to the wall term.

A working transonic PINN in the literature. Wassing, Langer and Bekemeyer [11] solve the Euler equations, energy equation included, for transonic NACA 0012 flow with a PINN. Their

method:

- collocation at the points of a 400×200 elliptic O-grid, with Fourier features of the grid indices;
- soft far-field, wall and periodicity terms, and a volume-weighted residual scaled by 2×10^4;
- L-BFGS alone, in mini-batches;
- artificial viscosity localised by an analytic shock sensor times a stagnation-point sensor.

They report Cp errors of 0.3–0.4 % at $M_\infty = 0.70$–0.75, and 3 % at 0.78. Without viscosity, or with global viscosity and no sensors, they capture no shock. They impose no explicit Kutta condition, yet their parametric network's lift agrees with the reference over $\alpha \in (0, 4)^\circ$ at $M_\infty = 0.72$. They train for 30–45 min per case, against 5.6 h for the `pure` runs here, on different hardware.

Their result bounds this study's conclusions about PINNs. The collapse of the Euler PINN and the low circulation of the potential PINN belong to the formulations and training tested here. Which of the many differences matters (residual weighting, input encoding, closure, optimiser, collocation and Mach range) is not known. The global viscous variants specified in this study's plan resemble their failing global form, though on a different formulation. They also conclude that PINNs do not currently beat CFD for a single forward case, which agrees with P1.

Cost. A per-case PINN costs between 88 and 220 SU2 solutions, depending on how GPU and CPU time are counted. At this problem size a forward-mode PINN is a learning exercise, not a solver; its strengths lie in inverse problems and sparse data, which this study did not test.

7.1 What the comparison does and does not show

The results invite the conclusion that neural operators suit aerodynamics better than PINNs. The study supports a narrower one. *For many-query prediction inside a well-sampled design space, when the CFD campaign can be paid for, a data-trained surrogate is the better tool, and a per-case PINN is not competitive even with the CFD it would replace.* On that task the margin is wide:

- in range, the FNO's near-field error is 1.3–5.3 times the floor, against 26–65 times for the PINN, whose lift is 64–101 % low on five of six cases (Tables 8 and 9);
- a new case costs 0.054 s with the FNO, against 5.6 h with the PINN, which is 88–220 SU2 runs (Table 7).

Five things keep this from being a general verdict on the two families.

1. **The contestants answered different questions.** The FNO learned from 881 SU2 solutions, about 64 CPU-hours of CFD. The PINN was given no data at all. A PINN solving one forward case that CFD already solves well is the PINN's least favourable use, which is why P1 was predicted.
2. **This study's PINN is not the state of the art.** The Euler PINN of Wassing, Langer and Bekemeyer [11] reaches 0.3–0.4 % Cp error on transonic NACA 0012 cases, in 30–45 min per case, with no explicit Kutta condition. The accuracy of the PINN here says more about the formulations tried than about PINNs. Even so, they reach the same conclusion on cost: a PINN does not currently beat CFD for a single forward case.
3. **The operator's win is narrower than it looks.**
    - It repays its training only after 1,126 new cases; for fewer, running SU2 is cheaper (Figure 11).

- POD, a linear baseline, beats it on in-range subcritical cases at 256×97. The FNO clearly earns its complexity only on supercritical cases, where shocks move with the inputs.
- Outside the training range it fails: 3.6 times the floor, with drag errors up to 181 counts, biased rather than scattered.
- Its shocks are smeared, and its subcritical near-field entropy falls below free stream on 9–24 of 37 cases, which SU2's never does.

4. **This is the easy end of aerodynamics.** The problem is two-dimensional, steady and inviscid, one family of sections with four inputs, on one structured grid. Practical aerodynamics is three-dimensional, viscous and turbulent, over varied geometry. There each training solution costs hours, a thousand of them is a real expense, and an operator must generalise across shapes and meshes. None of that was tested.
5. **The settings where PINNs are argued to be strong were not tested.** These are inverse problems, sparse measurements such as surface pressure taps, and parametric PINNs: one network trained across a range of inputs from the equations alone. The parametric network of Wassing et al. answers in 35 ms after 1.5–2.75 h of training. It amortises its cost the way an operator does, without needing a CFD campaign. Physics-informed neural operators, which add the equation residual to an operator's training, were left as an extension.

Which approach suits a given aerodynamic task therefore depends on three questions that this study answers for only one setting:

- how many queries are needed (against the break-even of Equation (28));
- whether the queries stay inside the sampled range;
- whether training data can be afforded at all.

All the accuracy figures are also relative to SU2 at 512×193, which on the anchor case is itself 6–8 % low in lift (Table 2). That leaves the ranking unchanged but limits any claim about absolute accuracy.

7.2 Limitations

- The costs mix hardware: SU2 and POD on the CPU, the FNO on the Apple GPU, and the PINN on both.
- The PINN results are single runs on three cases per regime.
- The POD selection sits on the edge of its search grid.
- The three SU2 grids are not in the asymptotic range, so the floor is a grid difference, not an extrapolated error. On the transonic anchor the truth itself is 6–8 % low in lift against a grid-converged reference (Table 2).
- The 100-chord outer boundary truncates C_L at the 10^{-3} level, equally for every method.
- The PINN's learned Γ was not recorded during training; it is inferred from the scored lift.

8 Open items and reproducibility

Still open.

- Which differences from Wassing, Langer and Bekemeyer's Euler PINN [11] would rescue this study's Euler PINN, and whether a localised viscosity would fix the potential PINN's circulation.
- Why this study's SU2 grids fall short of the grid-converged anchor values: section, far field, grid and scheme are not separated.
- The artificially viscous PINN runs, the expansion-shock test and an FNO ringing metric were not made.

Reproducing the work. Every stage is one resumable script, run from the repository root (Table 12). The committed records under `results/` are what every number here is read from. The SU2 cases, datasets and predictions (about 8 GB) are not committed but can be regenerated with those scripts. Figures 4, 6 and 7 read those regenerated files.

Table 12: The stage commands (SU2 found through `SU2_BIN_DIR`).

Stage	Command	Writes
S0	`python scripts/s0_check.py`	`results/s0/`
S1	`python scripts/s1_campaign.py train` (then `fine`)	`runs/su2/`
	`python scripts/build_dataset.py`	`data/hdf5/`, `results/s1/`
S2	`python scripts/s2_operator.py tune | final | pod`	`runs/predictions/`, `results/s2/`
S3a	`python scripts/s3_warmup.py`	`results/s3/warmup.json`
S3	`python scripts/s3_pinn.py -shard K/N`	`runs/predictions/`, `results/s3/`
S4	`python scripts/s4_score.py`	`results/s4/`
Figures	`python docs/report/make_figures.py`	`docs/report/figures/`

A Neural networks from the ground up

Appendices A to D explain the learned models of this study in enough detail to rebuild them. This appendix covers the building blocks they share. Appendix B covers the Fourier neural operator and Appendix C the physics-informed network, each exactly as implemented in `src/apno/models`. Appendix D sets the three models side by side. Throughout, $\boldsymbol{\theta}$ denotes all the trainable parameters of a model.

A.1 A network is a parameterised function

The simplest network, and the one the PINN uses, is the *multilayer perceptron* (MLP). It maps an input vector $\boldsymbol{x} \in \mathbb{R}^{n_0}$ to an output $\boldsymbol{y}$ by alternating affine maps with a fixed nonlinear function σ, applied element by element:

$$\boldsymbol{h}^0 = \boldsymbol{x}, \qquad \boldsymbol{h}^\ell = \sigma(\boldsymbol{W}^\ell \boldsymbol{h}^{\ell-1} + \boldsymbol{b}^\ell) \quad (\ell = 1, \dots, L), \qquad \boldsymbol{y} = \boldsymbol{W}^{L+1}\boldsymbol{h}^L + \boldsymbol{b}^{L+1}. \tag{29}$$

- The weight matrices $\boldsymbol{W}^\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and bias vectors $\boldsymbol{b}^\ell$ are the parameters, $n_\ell n_{\ell-1} + n_\ell$ of them per layer.
- L is the *depth*; n_ℓ is the *width* of layer ℓ.
- The PINN's network maps 3 inputs through $L = 7$ hidden layers of 96 units to one output. It has $(3 \cdot 96 + 96) + 6\,(96 \cdot 96 + 96) + (96 + 1) = 56{,}353$ parameters, plus the circulation Γ.

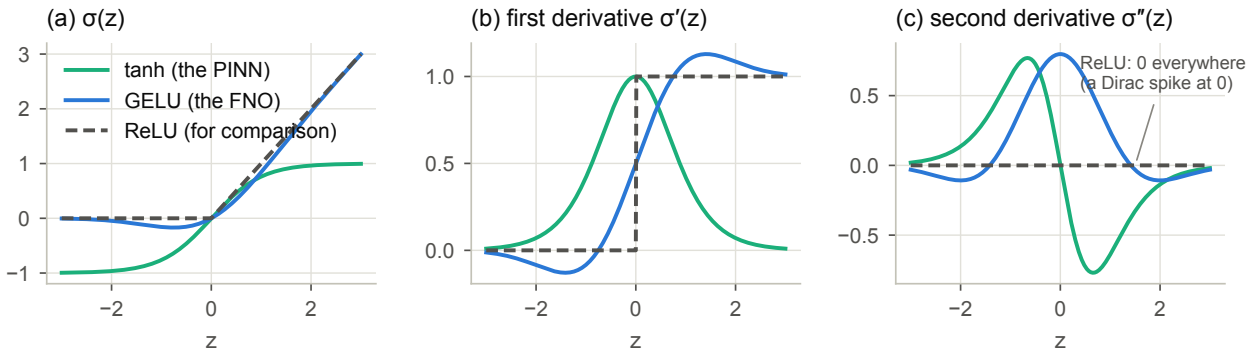


Figure 12: The activation functions used here and their derivatives. A PINN's loss contains derivatives of the network with respect to its *inputs*, so the activation's own derivatives appear in the equations being solved. A ReLU network's second derivatives are zero almost everywhere.

Why the nonlinearity matters. Without σ, Equation (29) collapses to a single affine map, however deep it is. With a nonlinear σ, a wide enough network can approximate any continuous function on a bounded set to any accuracy (the universal approximation property). It says nothing about how hard the right parameters are to *find*, which is the whole difficulty in practice.

Activation functions. Figure 12 compares the three activation functions relevant here.

- *tanh* is smooth and bounded, and has non-zero derivatives of every order.
- *GELU*, $z\,\Phi(z)$ with Φ the standard normal distribution function, is a smooth version of the ReLU.
- *ReLU*, $\max(z, 0)$, is piecewise linear: its second derivative is zero everywhere except at $z = 0$.

The choice matters for a PINN. A ReLU network is piecewise linear in its inputs, so every second derivative of its output is zero almost everywhere, yet the full-potential residual is built from second derivatives of the potential. This is why PINNs use tanh. The FNO only needs function values, not derivatives, so it uses GELU.

Initialisation. Training starts from random parameters. PyTorch draws each affine layer's weights and biases uniformly from $(-1/\sqrt{n_{\ell-1}},\ 1/\sqrt{n_{\ell-1}})$, which keeps each layer's initial outputs of moderate size whatever its width. Fixing the random seed makes a run repeatable.

A.2 Training is optimisation

The loss. A *loss* $\mathcal{L}(\boldsymbol{\theta})$ is a single number that measures how wrong the network is.

- In *supervised* learning, as for the FNO, it compares the network's output with known answers: here, SU2's fields for the training cases.
- In *physics-informed* learning, as for the PINN, there are no answers. The loss is how badly the network's output violates the governing equations and boundary conditions.

Training means finding parameters that make the loss small.

Gradient descent. The basic method steps against the gradient,

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \nabla_{\boldsymbol{\theta}} \mathcal{L}, \tag{30}$$

with a *learning rate* η.

Stochastic gradients. Computing the loss over all the data at every step is expensive, so the gradient is estimated from a small random subset:

- the FNO uses a *mini-batch* of 8 cases per step, and one pass over all the training cases is an *epoch*;
- the PINN uses one set of 20,000 random collocation points, redrawn every 1,000 steps.

The noise in such estimates is tolerated, and it can even help an optimiser leave poor regions.

A.3 Automatic differentiation

Reverse mode. Gradients are never derived by hand. PyTorch records every elementary operation of the forward computation as a graph and then applies the chain rule backwards through it (*reverse-mode automatic differentiation*, or backpropagation). The gradient of a scalar loss with respect to all parameters then costs a small multiple of one forward pass, however many parameters there are.

Derivatives with respect to the inputs. A PINN needs a second kind of derivative: of the network's *output* with respect to its *inputs*, such as $\partial\psi/\partial\xi$. These come from the same machinery. In the code,

```
torch.autograd.grad(f.sum(), xi, create_graph=True)
```

returns $\partial f/\partial\xi$ at every collocation point at once. Summing first is legitimate because each point's output depends only on that point's input, so the derivative of the sum with respect to the n-th input is the derivative of the n-th output.

Differentiating a derivative. The flag `create_graph=True` records the derivative computation itself as part of the graph, so the derivative can be differentiated again. This nesting is what PINNs are built from:

- the potential PINN differentiates ψ once for the velocity;
- it differentiates again for the divergence in the residual;
- training then differentiates the loss with respect to $\boldsymbol{\theta}$ through both levels.

Appendix C.4 counts what that costs.

A.4 Optimisers

Adam. Adam [4] keeps running averages of the gradient $\boldsymbol{g}_t$ and of its element-wise square, corrects their start-up bias, and scales each parameter's step by its own gradient history:

$$\boldsymbol{m}_t = \beta_1\boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t, \quad \boldsymbol{v}_t = \beta_2\boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta_t \frac{\boldsymbol{m}_t/(1-\beta_1^t)}{\sqrt{\boldsymbol{v}_t/(1-\beta_2^t)} + \epsilon}, \tag{31}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. Because every parameter gets its own step size, Adam copes with losses whose sensitivity varies by orders of magnitude between parameters. That is common in both networks here.

AdamW. AdamW [7], used for the FNO, adds *decoupled* weight decay: it also shrinks every weight by $\eta_t \lambda \boldsymbol{\theta}$ at each step ($\lambda = 10^{-4}$). This is separate from the gradient, which discourages large weights without distorting Adam's scaling.

Learning-rate schedules. Large steps make fast early progress; small steps settle into a minimum. Both networks lower η_t as training proceeds:

$$\text{FNO (cosine):}\quad \eta_e = \tfrac{1}{2}\eta_0\big(1+\cos(\pi e/E)\big), \qquad \text{PINN (exponential):}\quad \eta_k = \eta_0\,(\eta_1/\eta_0)^{k/K}. \tag{32}$$

Here e is the epoch of E (300 in the final runs), k the step of $K = 30{,}000$, and $\eta_0 = 10^{-3}$, $\eta_1 = 10^{-4}$.

L-BFGS. L-BFGS [6] is a quasi-Newton method, used to finish the PINN's training. It builds an approximation of the inverse Hessian from the last $m = 50$ pairs

$$\boldsymbol{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k, \qquad \boldsymbol{y}_k = \nabla\mathcal{L}_{k+1} - \nabla\mathcal{L}_k, \tag{33}$$

turns the gradient into a Newton-like search direction, and chooses the step length by a line search satisfying the strong Wolfe conditions. Near a minimum it converges far faster than Adam. It needs two things Adam does not:

- *A fixed objective.* If the collocation points changed between iterations, $\boldsymbol{y}_k$ would mix a change in the function with a change in its sampling, and the line search would chase a moving target.
- *Accurate arithmetic.* $\boldsymbol{y}_k$ is a difference of two nearly equal gradients, so in float32, with about 7 significant digits, it soon becomes mostly round-off. L-BFGS therefore runs in float64, on the CPU, because the Apple GPU has no float64.

A.5 Generalisation, and what it means for each model

A supervised model is judged on cases it never saw. The data are therefore split three ways:

- a *training set*, used to fit $\boldsymbol{\theta}$;
- a *validation set*, used to choose hyperparameters and when to stop;
- a *test set*, used once, for the final score.

Early stopping keeps the parameters from the epoch with the lowest validation loss, and stops when it has not improved for a set number of epochs (the *patience*). Training several *seeds* (initialisations and batch orders) shows how much of a result is luck.

A PINN has no training set and nothing to generalise to: it fits one case. Its only test is the comparison with SU2, which it never sees. The lesson of this study is that for a PINN a small loss is not the same as a right answer (Appendix C.8).

Questions

Q1. *Why could a ReLU network not serve as the PINN here?*
The full-potential residual contains second derivatives of the potential. A ReLU network is piecewise linear, so its second derivatives vanish almost everywhere, and the residual would not see the curvature that the equation constrains.

Q2. *Why does the L-BFGS phase keep one fixed set of collocation points?*
Its curvature pairs and its line search assume one deterministic function. Resampled points would make every gradient difference partly sampling noise.

Q3. *What does early stopping select, and from which data?*
The epoch with the lowest validation loss. Test cases are never used for any choice.

B The Fourier neural operator in detail

B.1 Learning an operator

An ordinary network maps vectors to vectors. A *neural operator* maps functions to functions. Here the input function is the geometry and free stream, given as six fields over the grid, and the output function is the flow state over the same grid:

$$\begin{aligned} \mathcal{G}: \quad & \boldsymbol{a}(i,j) = (\text{position},\ \text{ring index},\ M_\infty,\ \cos\alpha,\ \sin\alpha) \\ \longmapsto \quad & \boldsymbol{y}(i,j) = \text{the four targets of Equation (15).} \end{aligned} \tag{34}$$

The operator is learned from 881 examples of the pair $(\boldsymbol{a}, \boldsymbol{y})$. Ideally its parameters do not depend on the grid, so a model trained at one resolution can be evaluated at another; the measured resolution transfer error tests how far that holds here.

B.2 From kernel integrals to Fourier multipliers

A neural-operator layer replaces the matrix of an MLP by an integral operator acting on the hidden function $\boldsymbol{h}(\boldsymbol{z})$:

$$\boldsymbol{h}_{\ell+1}(\boldsymbol{z}) = \sigma\Big(\boldsymbol{W}\boldsymbol{h}_\ell(\boldsymbol{z}) + \int \kappa_{\boldsymbol{\vartheta}}(\boldsymbol{z}-\boldsymbol{z}')\,\boldsymbol{h}_\ell(\boldsymbol{z}')\,\mathrm{d}\boldsymbol{z}'\Big). \tag{35}$$

If the kernel κ depends only on $\boldsymbol{z}-\boldsymbol{z}'$, the integral is a convolution. The convolution theorem then turns it into a product in Fourier space,

$$(\kappa * \boldsymbol{h})(\boldsymbol{z}) = \mathcal{F}^{-1}\big[\hat{\kappa}(\boldsymbol{k})\,\hat{\boldsymbol{h}}(\boldsymbol{k})\big](\boldsymbol{z}). \tag{36}$$

The FNO [5] learns $\hat{\kappa}$ directly: one complex $w \times w$ matrix $\boldsymbol{R}_{\boldsymbol{k}}$ for each retained wavenumber $\boldsymbol{k}$, where w is the number of hidden channels. High wavenumbers are dropped. On a grid, $\mathcal{F}$ is the fast Fourier transform, so the layer costs $O(N \log N)$ for N nodes, and it couples every node to every other in a single layer.

B.3 The Fourier layer as implemented

The hidden state is a tensor $\boldsymbol{h} \in \mathbb{R}^{B\times w\times N_I\times(N_J+P)}$: batch, channels, nodes around the airfoil, and nodes outward including the padding. One Fourier layer (Figure 13, bottom) does five things.

1. *Transform*: $\hat{\boldsymbol{h}} = \mathrm{rfft2}(\boldsymbol{h})$. This is a full FFT along i and a real FFT along j, which keeps only the non-negative j-frequencies, since the negative ones are their complex conjugates.
2. *Truncate and mix*: keep i-frequencies $k_i \in \{0,\dots,23\} \cup \{-24,\dots,-1\}$ and j-frequencies $k_j \in \{0,\dots,7\}$. At each kept mode, mix the channels with that mode's own complex matrix,

$$\hat{o}_{b,o,\boldsymbol{k}} = \sum_{c=1}^{w} \hat{h}_{b,c,\boldsymbol{k}}\, R_{c,o,\boldsymbol{k}}, \tag{37}$$

   and set every other mode to zero. Positive and negative k_i need separate weight tensors (`w_pos` and `w_neg`), because the real FFT halves only the j axis.

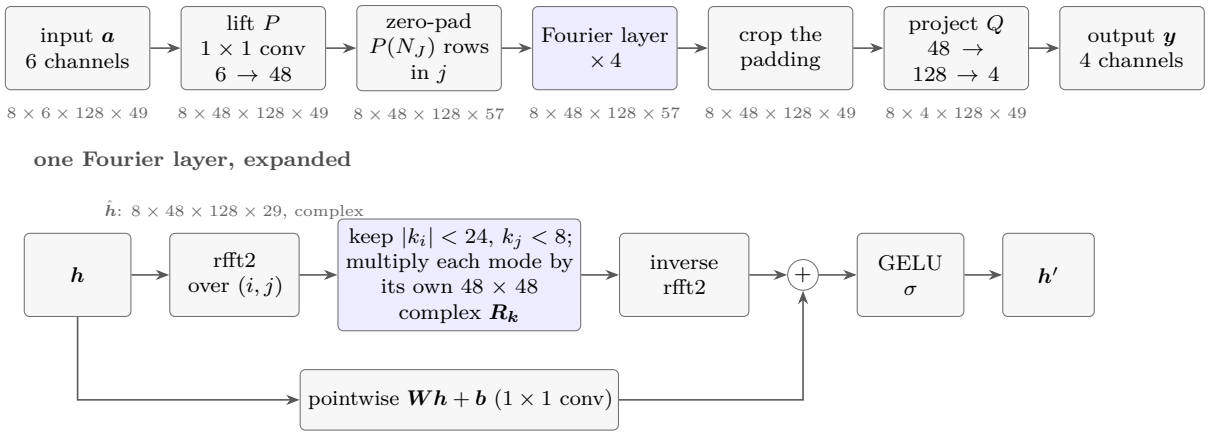


Figure 13: The FNO as implemented, with tensor shapes (batch, channels, i, j) for a batch of 8 at the 128×49 training resolution, with the selected width of 48. The projection is two 1×1 convolutions with a GELU between them. At 256×97 the same weights see $8 \times 48 \times 256 \times 114$ after padding, and the same 48×8 retained modes. Bottom: one Fourier layer.

Table 13: Parameters of the selected FNO (width 48, modes 24×8, 4 layers). A complex weight counts as two real numbers.

Part	Shape	Per part	Real numbers
Lift P	48×6 weights $+ 48$ biases		336
Spectral weights $\boldsymbol{R}$, 4 layers	$2 \times (48 \times 48 \times 24 \times 8)$ complex per layer	1,769,472	7,077,888
Pointwise $\boldsymbol{W}$, 4 layers	$48 \times 48 + 48$ per layer	2,352	9,408
Projection Q	$(48 \times 128 + 128) + (128 \times 4 + 4)$		6,788
Total			7,094,420

3. *Inverse transform* back to the grid.
4. *Pointwise path*: add $\boldsymbol{W}\boldsymbol{h} + \boldsymbol{b}$, a 1×1 convolution, which mixes channels at each node with no spatial coupling.
5. *Activation*: apply GELU, except after the last layer.

Why the pointwise path is needed. The spectral path can carry only 48×8 modes. The pointwise path passes everything else, the fine detail that nonlinearities create in particular, and it is not tied to periodicity at all. Without it, every layer's output would be a smooth, band-limited field.

Where the parameters are. Table 13 counts the parameters. Almost all of them, 99.8 %, are spectral weights. The model has about 7.1 million real parameters, and the training set holds $881 \times 4 \times 128 \times 49 = 22.1$ million target values. Weight decay, early stopping and the truncation itself are what keep it from memorising them.

B.4 Working in the O-grid's index space

Index space, not physical space. The FFT treats the grid as a uniform rectangle of nodes. The physical grid is curved and strongly stretched: its first wall spacing is 2×10^{-3} chords, while near

the outer boundary the spacing is chord-sized. A convolution in index space is translation-invariant in (i, j), not in (x, y), so the geometry reaches the model only through the position channels. The model has to learn what an index-space pattern means at each physical location.

Around the airfoil, periodicity is exact. Node $i = N_I$ is node $i = 0$, so the FFT along i involves no approximation. Mode k_i is k_i cycles per circuit of the airfoil at any N_I. This is what lets the same $\boldsymbol{R_k}$ mean the same thing at 128 and at 256 nodes.

Outward, the field is not periodic. A plain FFT along j would wrap the outer boundary onto the wall. Zero-padding by P rows puts a buffer between them. The discrete period in η is $(N_J + P)\,\Delta\eta = (N_J + P)/(N_J - 1)$, and for a learned radial mode to mean the same physical frequency at both resolutions that period must agree:

$$P = \text{round}\big(\tfrac{3}{16}(N_J - 1)\big) - 1 : \qquad \frac{49 + 8}{48} = \frac{97 + 17}{96} = \frac{19}{16}. \tag{38}$$

The padded rows are zero only on entry. The layers write into them, and they are cropped once, before the projection.

The transfer is still only first-order. At the end of the padding the field jumps back to the wall row's values, and that jump aliases differently at the two resolutions. The measured transfer error is therefore not zero: on the validation cases, the selected model's prediction at 128×49 differs from its prediction at 256×97 by a relative L2 of 0.035. In the test scores this is the factor of four between the two grids on in-range subcritical cases (Table 8).

B.5 Inputs and outputs

Inputs. The six input channels are given in Section 4. The position channels are $(x - \frac{1}{2},\ y)\ \ln(1 + r)/r$. This keeps the direction of each node from mid-chord and compresses its distance logarithmically: the outer circle at $r = 100$ maps to radius $\ln 101 \approx 4.6$, while the section is barely distorted. Without the compression, the far field would dominate the coordinate values and the near-body detail would occupy a sliver of their range.

Outputs. The targets (Equation (15)) are each standardised per channel, over all training cases and nodes: $z = (y - \mu_c)/s_c$. The network predicts z. A prediction is mapped back by inverting the standardisation and then Equation (15):

$$\hat{u} = y_1 + \cos\alpha, \quad \hat{v} = y_2 + \sin\alpha, \quad \hat{p} = 1 + \tfrac{1}{2}\gamma M_\infty^2 y_3, \quad \hat{\rho} = 1 + \tfrac{1}{2} M_\infty^2 y_4. \tag{39}$$

B.6 Training

Algorithm 1 is the training loop as run. At $n = 896$ the training set is the 881 converged cases among the first 896 Sobol points. An epoch is 111 mini-batches of 8, so 300 epochs are 33,300 AdamW steps, which took 36 minutes on the Apple GPU.

Figure 14 shows the validation loss for each training-set size. Every curve still improves at epoch 300: the best epochs are 274–297. The noise on each curve is the mini-batch noise, filtered by the decaying learning rate. More data lowers the whole curve, but by less at each doubling (Table 11).

Algorithm 1 FNO training (`apno.models.train_fno.train`)

Require: training cases (the first n Sobol points that converged), validation cases, seed
fit the per-channel standardiser on the training targets
$\boldsymbol{X}, \boldsymbol{Z} \leftarrow$ features and standardised targets at every second node (128×49)
initialise $\boldsymbol{\theta}$ from the seed; $v^* \leftarrow \infty$
for epoch $e = 0, \ldots, E-1$ **do**
    for each mini-batch b of 8 cases, in a fresh random order **do**
        $\mathcal{L} \leftarrow \mathcal{L}_A$ (or $\mathcal{L}_B$) of $\mathrm{FNO}_{\boldsymbol{\theta}}(\boldsymbol{X}_b)$ against $\boldsymbol{Z}_b$
        $\boldsymbol{\theta} \leftarrow \mathrm{AdamW}(\boldsymbol{\theta}, \nabla_{\boldsymbol{\theta}}\mathcal{L})$
    end for
    lower the learning rate (cosine); $v \leftarrow$ validation loss
    if $v < v^*$ **then**
        save $\boldsymbol{\theta}$ as the checkpoint; $v^* \leftarrow v$; $e^* \leftarrow e$
    else if $e - e^* \geq 30$ **then**
        stop
    end if
end for
return the checkpoint (the epoch of lowest validation loss)

B.7 What truncation does to a shock

A truncated Fourier series of a function with a jump overshoots near the jump and oscillates on both sides: the Gibbs phenomenon, an overshoot of about 9 % of the jump that does not shrink as modes are added. Figure 15 applies the FNO's mode budget directly to the wall pressure of the strongest showcase shock: the 24 lowest i-modes of each sign, out of 64, on the 128-node training wall. The shock becomes a ramp with ripples on both sides.

The FNO is not a single filter. The pointwise paths and the nonlinearities generate the higher frequencies that a filter alone would discard, so its own shocks are sharper than Figure 15(a). The bias remains visible in the scores, though: its in-range shock overshoot has median 0.42, against the floor's 0.19 (Table 10).

B.8 Evaluation, and keeping the test set out

Prediction. A prediction computes the six features on the case's own 256×97 grid, runs the network, and inverts the target mapping. It takes 0.054 s per case, about 4000 times faster than an SU2 run.

Leakage guard. The data loader refuses to read a test split. Test inputs are read through a separate function that returns only coordinates and free-stream values, never fields. A test case therefore cannot enter training, normalisation or model selection by accident.

Questions

Q1. *Why does a learned mode along i mean the same thing at 128 and 256 nodes, while the j direction needed a padding rule?*
Along i the grid is periodic with a period of one circuit at any N_I. Along j the period is set by the $N_J + P$ padded samples, and has to be made equal in η by choosing P.

Q2. *How many complex weights does one Fourier layer hold, and why two tensors?*
$2 \times 48 \times 48 \times 24 \times 8 = 884{,}736$. The real FFT halves only the j axis, so positive and negative

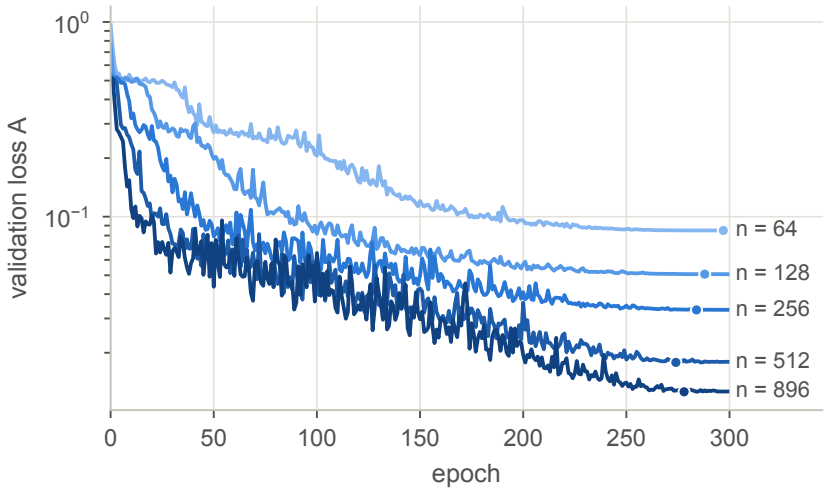


Figure 14: FNO validation loss per epoch for each training-set size n (seed 0, loss A), darker for larger n. Dots mark the epoch each run kept.

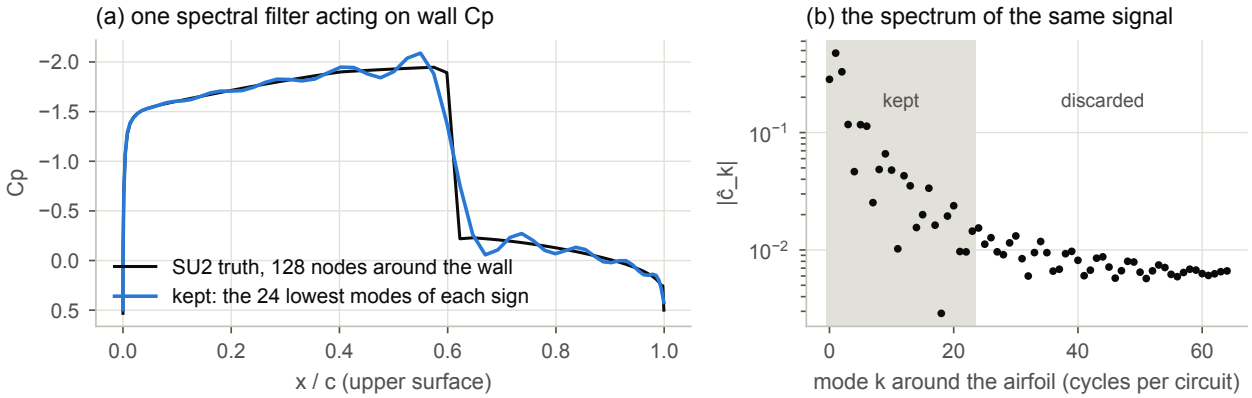


Figure 15: (a) Wall C_p on the upper surface of `test_in-0040` (SU2 truth at the 128×49 training resolution), and the same signal keeping only the FNO's 24 lowest i-modes of each sign. (b) The signal's Fourier amplitudes around the full wall. A shock puts energy into every mode, and a slowly decaying tail is what a jump looks like in a spectrum.

i-frequencies are distinct modes, each with its own weights.

Q3. *What was loss B meant to fix, and what happened?*
Forces depend only on wall pressure, while a field loss is dominated by the many off-wall nodes. Loss B adds the wall row with weight 10. In practice it was worse almost everywhere, and stopped early at epoch 81 (Section 6).

Q4. *Why is the FNO's in-range error four times smaller on its training grid?*
The resolution transfer is only first-order, because of the padding wrap; the same weights see differently aliased fields at 256×97.

C The physics-informed neural network in detail

C.1 The idea

A PINN [8] represents the solution of *one* problem as a network: a function of continuous coordinates. It is trained so that the governing equations and boundary conditions hold at many sampled points.

- There is no training data. The equations supply the supervision, so the method can in principle solve a case nobody has solved.
- Derivatives are exact, by automatic differentiation, and there is no mesh to generate.
- The answer can be evaluated anywhere, at any resolution.

The price is that every new case is a fresh, non-convex optimisation, and its loss is a residual, not an error.

C.2 The network and its inputs

The network is the MLP of Equation (29) with 7 hidden layers of 96 tanh units. Its three inputs are $(\cos 2\pi\xi,\ \sin 2\pi\xi,\ \eta)$, not (x, y) and not (ξ, η). Because ξ enters only through $\cos 2\pi\xi$ and $\sin 2\pi\xi$, the output satisfies $\psi(\xi + 1, \eta) = \psi(\xi, \eta)$ exactly: periodicity around the airfoil is built in, not learned. The full-potential network outputs one scalar ψ and has one more parameter, Γ, for 56,354 in all. The Euler network outputs three, $(\ln\hat{\rho},\ \hat{u} - \cos\alpha,\ \hat{v} - \sin\alpha)$, and has 56,547.

Why the circulation needs its own parameter. Around a lifting airfoil the potential is multivalued: one circuit around the body adds Γ to ϕ. A network of $(\cos 2\pi\xi, \sin 2\pi\xi, \eta)$ is single-valued and cannot represent that jump. The term $\Gamma\Theta_v$ carries it instead: Θ_v jumps by exactly one across the wake cut, and its gradient, the velocity, is continuous there. The network therefore represents only the single-valued remainder.

C.3 Hard and soft constraints

Every condition the solution must meet is either built into the representation (*hard*), holding for any $\boldsymbol{\theta}$, or added to the loss (*soft*), holding only as well as the optimiser manages. Table 14 lists both formulations.

Each hard constraint removes a way to fail. Each soft one is something the optimiser may trade against the others. The Euler PINN's failure here is this distinction at work: its far field was soft, and the optimiser traded it away for fluid at rest. The full-potential PINN makes the far field hard and puts the wall in the loss instead. A soft far field is not fatal in itself, though: Wassing, Langer and Bekemeyer [11] keep it soft, weight the residual 2×10^4 times more heavily, and succeed.

C.4 Derivatives through the grid map

The chain rule through the map. The network is a function of (ξ, η), but the equations are written in (x, y). The map Equation (8) links them, and its derivatives (the *metric terms*) come from automatic differentiation of the same function the mesh generator uses. By the chain rule,

$$\begin{pmatrix} \phi_x \\ \phi_y \end{pmatrix} = \frac{1}{J} \begin{pmatrix} y_\eta & -y_\xi \\ -x_\eta & x_\xi \end{pmatrix} \begin{pmatrix} \phi_\xi \\ \phi_\eta \end{pmatrix}, \qquad J = x_\xi y_\eta - x_\eta y_\xi. \tag{40}$$

Table 14: How each condition enters the two PINN formulations.

Condition	Euler PINN	Full-potential PINN
Periodic around the airfoil	hard: inputs $\cos 2\pi\xi$, $\sin 2\pi\xi$	hard: the same
Far field	soft: $(\hat{\rho}-1)^2 + \lvert\hat{\boldsymbol{u}}-\hat{\boldsymbol{u}}_\infty\rvert^2$ on 1,000 outer points, weight 10	hard: $\psi(1-\eta)=0$ on $\eta=1$
Wall tangency	hard: normal component removed with weight $(1-\eta)^4$	soft: $(\boldsymbol{u}\cdot\hat{\boldsymbol{n}}_w)^2$ at 512 wall points, weight 10
Positive density	hard: $\hat{\rho}=\exp(\cdot)$	hard: Bernoulli, base floored at 10^{-6}
Constant total enthalpy	hard: pressure from H_∞	hard: Bernoulli at H_∞
Irrotational, isentropic	not imposed	hard: $\boldsymbol{u}=\nabla\phi$, $\hat{p}=\hat{\rho}^{\,\gamma}$
Momentum	residual (two equations)	identity
Continuity	residual	residual (the only equation)
Kutta condition	soft: $\lvert\boldsymbol{u}\rvert^2=0$ at the trailing edge, weight 10	soft: equal speeds at $\xi=\delta, 1-\delta$, weight 10
Circulation	implied by the velocity field	explicit parameter Γ

The divergence. The divergence of the mass flux $(f,g)=\rho(u,v)$ uses the strong conservation form Equation (19). It is the same quantity, rearranged so that the metric terms sit inside the derivatives, as in a finite-volume scheme. The test suite checks that uniform flow gives zero to round-off through this operator on the curved grid.

The trailing edge. At the trailing-edge wall point $J=0$, because $x_\xi = y_\xi = 0$ there. The velocity there is 0/0, and the code reports the Kutta stagnation value, zero, at that single point.

The cost of the derivatives. For one training step of the full-potential PINN, the derivatives nest as follows:

1. a forward pass gives ψ and hence ϕ at 20,000 points;
2. a first differentiation gives ϕ_ξ and ϕ_η, hence u, v, ρ and (f,g);
3. a second differentiation, of $y_\eta f - x_\eta g$ with respect to ξ and of $-y_\xi f + x_\xi g$ with respect to η, gives the residual $\mathcal{R}$, which involves second derivatives of ψ;
4. backpropagation of the loss to $\boldsymbol{\theta}$ differentiates that whole graph once more.

In effect the step computes third-order derivatives of the network. The Euler PINN's residual needs only first derivatives of its outputs, because its fluxes are algebraic in the state. That is why one full-potential Adam step cost 0.88 s on the CPU against the Euler PINN's 0.60 s, and 0.19 s on the Apple GPU. Figure 16 draws the computation.

C.5 Collocation points

Where the points fall. Each draw is 20,000 points uniform in (ξ,η), which the map turns into points strongly clustered at the body (Figure 17). In the first draw of the production run on `test_in-0022`, 11,018 points lie within 2.5 chords of mid-chord, a disk that is 0.06 % of the domain's area. Two consequences follow:

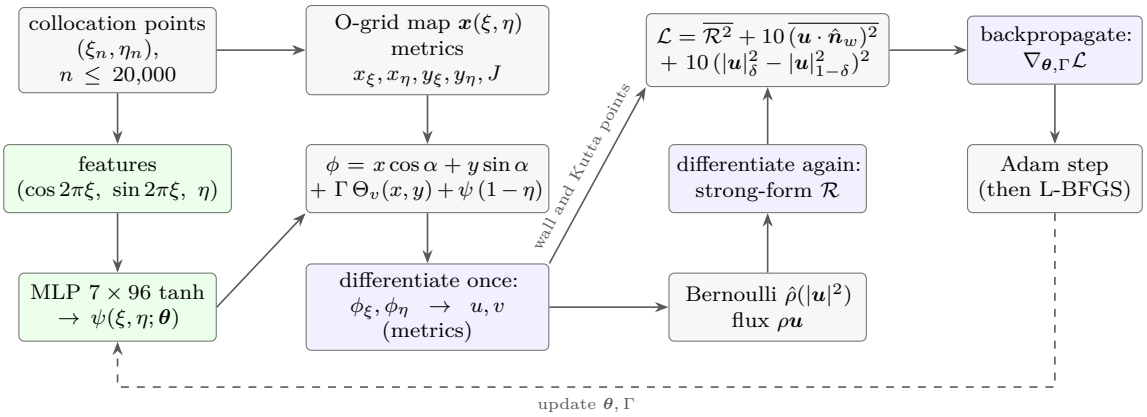


Figure 16: One training step of the full-potential PINN. Green: the network. Blue: automatic differentiation. The residual needs two nested derivatives of the network with respect to its inputs, and the update differentiates through both.

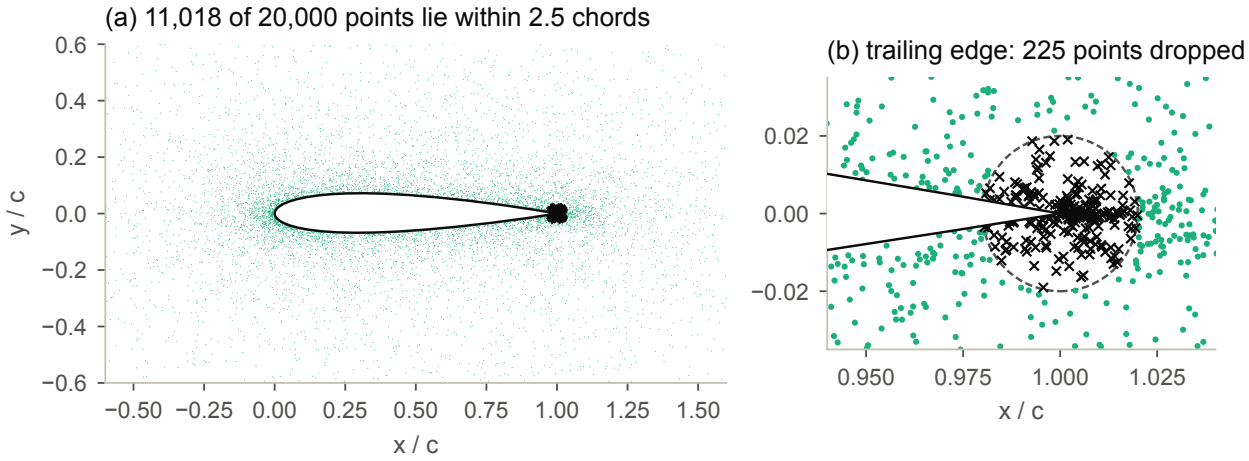


Figure 17: The first draw of collocation points of the `pure` run on `test_in-0022` (seed 0, as drawn in production). Points uniform in (ξ, η) crowd the section and its wake line. Crosses: points inside the 0.02-chord trailing-edge exclusion (dashed circle), which are dropped from the residual.

- A mean over these points weights each physical region by $1/J$. That favours the near field where the answer matters, but it also means the loss barely sees most of the domain.
- Points are redrawn every 1,000 Adam steps, so the network cannot satisfy the equation only at a fixed set of points and be wrong between them.

The trailing-edge exclusion. After each draw, the points within 0.02 chord of the trailing edge are dropped (225 in the draw shown). The corner flow is singular there, and those few points would otherwise dominate the loss.

Wall points. The 512 wall points sit at $\xi = (k + \frac{1}{2})/512$, half a spacing off the grid, so that none falls on the degenerate trailing-edge point.

Algorithm 2 PINN training for one case (`apno.models.train_pinn.train`)

Require: the case (M_∞, α, t, m) and its O-grid map; no flow data
initialise $\boldsymbol{\theta}$ from seed 0; $\Gamma \leftarrow 0$; weights $w_{\text{wall}} = w_K = 10$
for $k = 0, \ldots, 29{,}999$ **do** ▷ Adam, float32, Apple GPU
  if $k \bmod 1000 = 0$ **then**
    draw 20,000 points uniform in (ξ, η); drop those within 0.02 chord of the trailing edge
  end if
  evaluate $\mathcal{R}$, the wall term and the Kutta term; form $\mathcal{L}$ by Equation (24)
  if gradient-norm balancing **and** $k \bmod 100 = 0$ **then**
    update w_{wall}, w_K by Equation (25)
  end if
  $(\boldsymbol{\theta}, \Gamma) \leftarrow \text{Adam}\big((\boldsymbol{\theta}, \Gamma), \nabla\mathcal{L}\big)$; decay the learning rate exponentially
end for
move the network and the last point set to the CPU in float64
run L-BFGS on that fixed objective: up to 5,000 iterations, history 50, strong Wolfe line search
return the network evaluated at the 256×97 nodes

C.6 The loss and its weights

One trade-off for several conditions. The loss Equation (24) adds terms with different units and very different sizes and sensitivities. Minimising a weighted sum finds one point on a trade-off between them. Only if the network could satisfy every term exactly at once would the weights not matter, and with finite networks and a singular corner it cannot. With fixed weights of 10, the `pure` runs ended with residual 1–24×10^{-3}, wall term 0.5–21×10^{-4} and Kutta term below 6×10^{-6} (Table 5).

Gradient-norm balancing. Gradient-norm balancing [12] sets each weight from how strongly its term pulls on the parameters, compared with the residual. It updates the weights by Equation (25) every 100 steps, which raises the weight of any term whose gradient is small. The `gradnorm` run shows the risk. A term whose gradient is small *because it is already nearly satisfied* gets a huge weight: the Kutta weight reached 7×10^{10}. A term that stays stubbornly unsatisfied, the wall term here, then holds most of the objective, and L-BFGS spent its iterations on it at the residual's expense (Figure 9).

C.7 Optimisation

Algorithm 2 is the training procedure. The two phases play to each optimiser's strength:

- *Adam*, robust to noisy, poorly scaled gradients, makes the large early progress in float32 on the GPU, over fresh points every 1,000 steps;
- *L-BFGS* then converges locally on a fixed, deterministic objective in float64 on the CPU.

On the six `pure` runs the phases took similar time, 9,658 s of Adam and 10,544 s of L-BFGS on average. Finally the network is evaluated at the 256×97 grid nodes in float64, and the state is converted to the dataset's units, $p/p_\infty = \gamma M_\infty^2\, p/(\rho_\infty U_\infty^2)$.

C.8 What this study teaches about PINNs

1. **The formulation decides whether the minimum is the answer.** The Euler PINN's loss had a better minimum than the true flow: fluid at rest near the body, reaching the free

stream only at the outer circle. Here, changing the equations and the hard constraints fixed that. Wassing, Langer and Bekemeyer [11] reach a working Euler PINN by other means, with different weighting, inputs and optimiser, so the minimum is a property of the whole formulation, loss weights included, not of the Euler equations alone.

2. **Geometric singularities dominate.** At a sharp trailing edge the map is degenerate and the true flow is singular. Before the fixes, ten of 20,000 points carried 99.9 % of the initial residual.

3. **A small loss is not a right answer.** Every `pure` run ended with a residual of 10^{-3} to 2×10^{-2} and a Kutta term met to 6×10^{-6} or better, and five of the six had lift far too low. Without the Kutta term the loss was lower still (Figure 9(a)): without it every circulation gives an admissible potential flow, so the loss cannot tell them apart. Only a comparison with an independent solution exposes this.

4. **Automatic weighting is not automatically better.** Balancing by gradient norms handed the objective to the hardest term and wrecked the equation residual.

5. **Each case costs a full training.** The 56,354 parameters are refitted from scratch with third-order automatic differentiation on 20,000 points for 30,000 Adam steps and up to 5,000 L-BFGS iterations: about 5.6 hours here, against 3.5 minutes for SU2.

Questions

Q1. *Which conditions are exact in the full-potential PINN, and which are penalties?*
Periodicity, the far field, positive density, constant total enthalpy, irrotationality and isentropy are exact. Continuity, wall tangency and the Kutta condition are in the loss (Table 14).

Q2. *Why does a single-valued network need the extra term $\Gamma\Theta_v$?*
A lifting flow's potential jumps by Γ around the body. The network's inputs make it single-valued, so the jump must be supplied by a term that carries it, with a continuous gradient.

Q3. *Why is a full-potential Adam step dearer than an Euler one?*
Its residual already contains second derivatives of the network, so backpropagation works through third-order terms. The Euler residual needs only first derivatives.

Q4. *The `nokutta` run reached a lower loss than `pure`. Why is it not better?*
Without the Kutta term the problem has a family of solutions, one per circulation, and all of them satisfy the remaining terms equally well. The loss cannot select the physical one, and neither run found it.

D The three models side by side

Table 15 summarises the comparison.

- The two data-driven models are cheap to query because the expensive part, the SU2 campaign, is paid once.
- The PINN avoids that campaign entirely, and pays a whole optimisation for every case instead.
- The data-driven models know only what the data showed them, and they fail when asked to extrapolate.
- The PINN knows the equations but not, unaided, which of their solutions is the physical one.

Table 15: The learned models compared, with this study's figures.

	FNO	POD + RBF	PINN (full potential)
What it learns	an operator: inputs $\mapsto$ fields, for the whole family	a linear subspace of fields, and smooth weights over the inputs	one solution, as a continuous function
Data needed	881 SU2 solutions	881 SU2 solutions	none: the equations
Fitted parameters	7.09 M real, by AdamW	256 modes of 99,328 values (25 M numbers) plus RBF weights, by linear algebra	56,354, by Adam then L-BFGS
One-off cost	campaign + 4,867 s tuning + 2,160 s training (GPU)	campaign + 351 s search and fit (CPU)	none
Cost per new case	0.054 s	0.007 s	5.6 h (wall)
Physics built in	none: everything is learned	none	continuity solved; far field, isentropy and irrotationality exact
Geometry	node coordinates as input channels	implicit in the fields	through the map and its metric terms
Resolution	trained at 128×49, evaluated at 256×97	fixed at 256×97	any (continuous)
Main failure here	resolution transfer; biased extrapolation; smeared shocks	displaced, stepped shocks; non-physical states in extrapolation	circulation far too low; equation unenforced under gradnorm

References

[1] I. H. Abbott and A. E. von Doenhoff. *Theory of Wing Sections*. Dover, New York, 1959.

[2] T. D. Economon, F. Palacios, S. R. Copeland, T. W. Lukaczyk, and J. J. Alonso. SU2: An open-source suite for multiphysics simulation and design. *AIAA Journal*, 54(3):828–846, 2016.

[3] A. Jameson, W. Schmidt, and E. Turkel. Numerical solution of the Euler equations by finite volume methods using Runge–Kutta time-stepping schemes. AIAA Paper 81-1259, 1981.

[4] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[5] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

[6] D. C. Liu and J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45:503–528, 1989.

[7] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[8] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[9] L. Sirovich. Turbulence and the dynamics of coherent structures. *Quarterly of Applied Mathematics*, 45(3):561–571, 1987.

[10] J. C. Vassberg and A. Jameson. In pursuit of grid convergence for two-dimensional Euler solutions. *Journal of Aircraft*, 47(4):1152–1166, 2010. doi:10.2514/1.46737.

[11] S. Wassing, S. Langer, and P. Bekemeyer. Physics-informed neural networks for transonic flows around an airfoil. arXiv:2408.17364v3 [physics.flu-dyn], 2025.

[12] S. Wang, Y. Teng, and P. Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.